



DANMARKS TEKNISKE UNIVERSITET

Tensor Networks for Reinforcement Learning

(02466) PROJECT WORK - BACHELOR OF ARTIFICIAL
INTELLIGENCE

Authors:

Oliver Christensen (s246176)
Bertram Kjærholm Foldberg Bendsen (s244829)
Frederik Boye Vesth (s246199)

Abstract: Throughout this project, various implementations of tensor networks in reinforcement learning were tested, specifically on 3 different grid-based environments based on the Minigrid environment series by Farama. 5 different tensor based environments, being pure CP Q-table approximation, CP and Tucker weight decomposition, tensorized input embedding and an MPS tensor network, was compared to a baseline Multilayer Perceptron Neural Network.

It was found that tensor networks can substantially reduce parameter amount by up to 29x when using CP-based models instead of tabular Q-learning. Furthermore, up to 350x improvement was shown when using architectures that preserve the spatial structure of the input, specifically tensorized layers and TT input embedding, compared to baseline MLP models.

The key finding is that preserving the spatial features of the input space is much more decisive for model performance than the choice of tensor decomposition method. In addition to this, benefits in tensor model efficiency are shown to scale with size, proving better results at larger state spaces with more dimensions.

Section	Responsible:		
	(s246176)	(s244829)	(s246199)
Introduction	40%	30%	30%
Method: Theoretical	30%	40%	30%
Method: Experimental Setup	30%	30%	40%
Results	40%	30%	30%
Discussion	30%	40%	30%

June 24, 2026

Contents

List of Abbreviations	3
Introduction	4
Why reinforcement learning is important	4
The importance of optimizing information usage in RL	4
Brief explanation of tensor networks and its applications	4
Research questions and operationalization	4
Brief overview of report	5
Method: Theoretical	5
Tensors	5
Mathematical definition of tensors and notation	5
Decomposition methods	7
CP/Parafac	7
Tucker	8
Tensor Train (TT)	8
Comparison of decomposition methods	9
Tensor diagrams	9
Reinforcement learning	9
Tabular Q-learning	10
Deep Q-learning	10
Method: Experimental Setup	11
Overview of experimental approach	11
Environments	11
Data generation	12
Data buffer	12
Environment: MiniGrid-Empty	12
Environment: MiniGrid-DoorKey and MiniGrid-DistShift	13
Environment: Custom 8D and 10D	13
Environment: Minigrid-variations	13
Fixed start vs. random start	13
Experimental Protocol	14
Evaluation Metrics	14
Libraries and code structure	15
Code structure	15
Gymnasium	15
Torch	15
Tensorly	15
Phase 1: Tabular Q-learning setup	15
CP Q-table representation and update	16
Computational efficiency	17
Phase 2: Deep Q-Learning setup	18
designs/ explained	18
Other versions	20
Results	20
Phase 1: Tabular Q-learning Experiment	20
Accuracy	20
Learning rate	20

Efficiency	20
Phase 2: Deep Q-learning Experiment	21
Discussion	26
General discussion on results and further improvements	26
Where Tensor models shine	27
The deadly triad	28
Training time	28
Ethical considerations	28
Conclusion	29
Appendix	31
GitHub link for code	31
cases/ directory explained	31
More results	36
Cross-design best solve rates	36
Baselines across environments	36
Case 0a: DoorKey-5x5 1000 episodes output	37
CP Q-function: learning curves (Empty-5x5)	37
CP Q-function: parameter efficiency (Empty-5x5)	38
CP Q-function: learning curves (DoorKey and DistShift)	39
Weight decomposition: parameter efficiency (DistShift1)	40
Weight decomposition: convergence heatmap (DistShift1)	41
Weight decomposition: parameter efficiency (DoorKey-5x5)	42
Weight decomposition: parameter efficiency (Empty-5x5)	43
Input reduction: convergence heatmap (DistShift1)	44
MPS: convergence heatmap (Empty-5x5)	45
Parameter counts for all designs	45

List of Abbreviations

AI	Artificial Intelligence
ALS	Alternating Least Squares
CANDECOMP	CANonical DECOMPosition
CNN	Convolutional Neural Network
CP	CANDECOMP/PARAFAC (tensor decomposition)
DQN	Deep Q-Network
HOOI	Higher-Order Orthogonal Iteration
MLP	Multi-Layer Perceptron
MPS	Matrix Product State
NLMS	Normalised Least-Mean-Squares
PARAFAC	PARAllel FACtor analysis
PCA	Principal Component Analysis
RAM	Random Access Memory
RL	Reinforcement Learning
SDG	Sustainable Development Goal
SVD	Singular Value Decomposition
TD	Temporal Difference
TN	Tensor Network
TT	Tensor Train
TT-SVD	Tensor Train via Singular Value Decomposition
TUCKALS3	Tucker Alternating Least Squares (3-way)

Introduction

Artificial intelligence has experienced rapid growth in recent decades, with machine learning systems becoming a large component to a wide range of technologies and industries. As AI models are increasingly deployed in real-world applications, the ability to learn efficiently from limited data and to operate in dynamic, uncertain environments has become crucial. Among the major branches of machine learning, reinforcement learning (RL) distinguishes itself through its focus on sequential decision-making and its ability to autonomously learn strategies through interaction with an environment.

Why reinforcement learning is important

Reinforcement learning has evolved from early theoretical foundations in the mid-20th century to one of the most active research areas in modern AI. Its core strength lies in its capacity to learn optimal actions in unfamiliar or complex settings through trial-and-error. This makes RL particularly valuable in domains such as robotics, autonomous systems, healthcare, and finance, where adaptive behavior is essential [8]. However, despite these successes, RL remains challenging to scale. Standard RL pipelines often require substantial amounts of data, long training time, and large neural network architectures. Models can become prohibitively memory-heavy, and their sample inefficiency limits their usability in scenarios where data collection is expensive or physically constrained.

The importance of optimizing information usage in RL

As RL environments grow in complexity, so does the need for more compact and structured model representations. Many recent approaches rely on deep neural networks with millions of parameters, which can lead to overfitting, poor generalization, and inefficient training. Reducing the amount of information required—both in terms of parameters and training data—can therefore significantly improve the practicality of RL algorithms. This raises the question of whether alternative function representations can offer improved efficiency without sacrificing performance.

Brief explanation of tensor networks and its applications

Tensor networks (TNs) originate from quantum many-body physics [6], where they were developed to represent extremely high-dimensional systems in a compressed and structured way. Models such as CANDECOMP/PARAFAC Tensor Decomposition (CP), Tensor Train (TT) and Tucker decomposition offer efficient representations of large state spaces by exploiting low-rank structure and local correlations. In recent years, these techniques have gained traction in classical machine learning for tasks such as supervised learning, generative modelling, and neural network compression. Their promise lies in their ability to capture complex relationships with far fewer parameters than traditional deep learning models. Despite this, the exploration of tensor networks within reinforcement learning remains limited, and it is still unclear how well TN-based models can perform in sequential decision-making tasks. This gap in publications provides the central motivation for the present study.

Research questions and operationalization

The project aims to answer:

Can tensor network-based Q -function approximators achieve competitive performance compared to standard neural networks while using fewer parameters?

How do different tensor network architectures compare in reinforcement learning tasks in terms of performance, parameter efficiency, and training stability?

To operationalize the first research question, we define competitive performance as achieving a solve rate within 10 percentage points of the baseline DQN on a given environment, while using at least 50% fewer parameters. This reflects the core motivation of the work: a tensor network model that matches the baseline exactly but uses the same number of parameters offers no practical benefit, while a model that is radically more parameter-efficient but solves the task far less reliably is equally unfulfilling. The 10-point margin is chosen to allow for the variance of RL training across seeds, while still demanding

comparable task performance. The 50% parameter reduction threshold is deliberately conservative, ensuring that only models with a meaningful structural advantage are considered competitive. For the second research question, architectures are compared along three axes: final solve rate (performance), solve rate per 1,000 parameters (parameter efficiency), and convergence speed measured as the number of training steps to reach 80% of the model's own final solve rate (training stability). A design is considered superior on an axis if it ranks first on that metric across at least two of the three test environments.

Brief overview of report

The remainder of this report is structured as follows. The section 'Method: Theoretical' provides the theoretical foundation, beginning with a formal introduction to tensors, their properties, and the decomposition methods central to this work - CP, Tucker, and Tensor Train - before introducing the reinforcement learning framework, covering tabular Q-learning, deep Q-learning, and the connection between the two that motivates a tensorized approach. The section 'Method: Experimental Setup' describes the experimental setup, detailing the environments used, the data generation strategy, and the five model designs evaluated across both the tabular and deep Q-learning phases. The 'Results' section presents the results of both phases, reporting on accuracy, learning speed, and parameter efficiency across all designs and environments. 'Discussion' discusses the key findings, including the role of spatial structure, the failure modes observed, and the computational trade-offs introduced by tensor representations, and reflects on the ethical implications of the work and its broader societal relevance. 'Conclusion' returns to the research questions and summarizes what the experiments established. Supporting figures, extended result tables, and appendix material are collected at the end.

Method: Theoretical

This section covers the methods used in this project and the general approach taken to implement them. Furthermore it also covers the theories behind the methods and the tools used to implement them.

Tensors

First off, a general introduction to tensors and their properties will be given.

A tensor is defined as a multi-dimensional array, which can be used to represent data with more than two dimensions. It can be thought of as a generalization of matrices to higher dimensions. In this way, tensors can represent complex data structures that cannot be easily represented using traditional matrices. Tensors are commonly used in chemometrics, signal processing, numerical linear algebra, computer vision, numerical analysis, data mining, etc; the strength being that they can be used to represent complex data structures such as images, videos, and text. [7, p. 1]

Mathematical definition of tensors and notation

Mathematically, a tensor can be defined as a multi-dimensional array of numerical values. The order of a tensor is the number of dimensions it has, and each dimension is called a mode. The size of the n -th mode is denoted I_n . For example, a 3D tensor with dimensions (2, 3, 4) has an order of 3 and modes of size 2, 3, and 4. Another way to think of order, is the amount of indexes needed to access an element in the tensor.

This can be written as follows:

$$\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N} \quad (1)$$

By following the notation used in [10, p. 3], we can denote the following elements:

- Scalars as lowercase letters (e.g. x , I , etc.)
- Vectors as bold lowercase letters (e.g. $\mathbf{x}$) which are first order tensors

- Matrices as bold uppercase letters (e.g. $\mathbf{X}$) which are second order tensors
- Higher-order tensors as calligraphic letters (e.g. $\mathcal{X}$)
- Indexes as lowercase subscript letters (e.g. $\mathbf{X}_{i,j}$, $\mathcal{X}_{i,j,k}$, etc. up to $\mathcal{X}_{i_1,i_2,\dots,i_N}$)
- Identity matrix ($\mathbf{I}$), transpose ($\mathbf{X}^T$), and inverse ($\mathbf{X}^{-1}$) as usual
- N : the order of the tensor, so N 'th order tensor has N modes, and N indexes to access an element in the tensor
- I_n : the size of the n -th mode of the tensor, so the n -th mode has I_n elements, also showed as $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, where I_n is the size of the n -th mode
- To access an row or column of a matrix, we can use the notation $\mathbf{X}_{i,:}$ for the i -th row and $\mathbf{X}_{:,j}$ for the j -th column. For higher-order tensors, we can use the notation $\mathcal{X}_{i_1,:,i_3,\dots,i_N}$ to access a slice of the tensor along the second mode, for example.

Furthermore, one needs to define the operations of tensors, such as addition, multiplication, etc. These operations can be defined in a way that generalizes the corresponding operations for matrices. For example, tensor addition can be defined as element-wise addition of two tensors of the same size, while tensor multiplication can be defined as a generalization of matrix multiplication to higher dimensions.

More expressively, the tensor operations are defined as follows [10, p. 4]: Tensor unfolding: rearranging the elements of a tensor into a matrix by fixing one mode and varying the others. This is also known as matricization or flattening. Mathematically, given a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the mode- n unfolding of $\mathcal{X}$ is denoted as $\mathbf{X}_{(n)}$ and has dimensions $I_n \times (I_1 I_2 \dots I_{n-1} I_{n+1} \dots I_N)$. This is the specific tensor unfolding method, where we unfold each layer into columns of a matrix.

Tensor vectorization: like tensor unfolding, but instead of rearranging the elements into a matrix, we rearrange them into a vector. This is also known as flattening. Mathematically, given a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the vectorization of $\mathcal{X}$ is denoted as $\mathbf{x}$ and has dimensions $(I_1 I_2 \dots I_N) \times 1$.

Kronecker product: Given two matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{p \times q}$, the Kronecker product of $\mathbf{A}$ and $\mathbf{B}$ is denoted as $\mathbf{A} \otimes \mathbf{B}$ and has dimensions $(mp) \times (nq)$. The Kronecker product is defined as the block matrix obtained by multiplying each element of $\mathbf{A}$ by the entire matrix $\mathbf{B}$. So,

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} & \cdots & a_{2n}\mathbf{B} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & a_{m2}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \quad (2)$$

Hadamard product: Given two matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{m \times n}$, the Hadamard product of $\mathbf{A}$ and $\mathbf{B}$ is denoted as $\mathbf{A} \circ \mathbf{B}$ and has dimensions $m \times n$. The Hadamard product is defined as the element-wise product of the two matrices. So,

$$\mathbf{A} \circ \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \cdots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \cdots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \cdots & a_{mn}b_{mn} \end{bmatrix} \quad (3)$$

Khatri-Rao product: Given two matrices $\mathbf{A} \in \mathbb{R}^{m \times R}$ and $\mathbf{B} \in \mathbb{R}^{n \times R}$ with the same number of columns, the Khatri-Rao product of $\mathbf{A}$ and $\mathbf{B}$ is denoted as $\mathbf{A} \odot \mathbf{B}$ and has dimensions $(mn) \times R$. It is defined as the column-wise Kronecker product:

$$\mathbf{A} \odot \mathbf{B} = [\mathbf{a}_1 \otimes \mathbf{b}_1 \quad \mathbf{a}_2 \otimes \mathbf{b}_2 \quad \cdots \quad \mathbf{a}_R \otimes \mathbf{b}_R] \quad (4)$$

where $\mathbf{a}_r$ and $\mathbf{b}_r$ denote the r -th columns of $\mathbf{A}$ and $\mathbf{B}$ respectively. The Khatri-Rao product appears in certain formulations of the CP decomposition. [7, p. 462]

Tensor outer product: Given two tensors $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ and $\mathcal{B} \in \mathbb{R}^{J_1 \times J_2 \times \dots \times J_M}$, the tensor outer product of $\mathcal{A}$ and $\mathcal{B}$ is denoted as $\mathcal{A} \circ \mathcal{B}$ and has dimensions $I_1 \times I_2 \times \dots \times I_N \times J_1 \times J_2 \times \dots \times J_M$. This results in a tensor of order $N + M$. Each element of the outer product is simply the product of the corresponding elements:

$$(\mathcal{A} \circ \mathcal{B})_{i_1, \dots, i_N, j_1, \dots, j_M} = \mathcal{A}_{i_1, \dots, i_N} \cdot \mathcal{B}_{j_1, \dots, j_M} \quad (5)$$

For example, the outer product of two vectors $\mathbf{a} \in \mathbb{R}^I$ and $\mathbf{b} \in \mathbb{R}^J$ yields a matrix of dimensions $I \times J$, with entries $(\mathbf{a} \circ \mathbf{b})_{i,j} = a_i b_j$. [7, p. 458]

N-mode product: Given a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ and a matrix $\mathbf{U} \in \mathbb{R}^{J \times I_n}$, the n -mode product of $\mathcal{X}$ with $\mathbf{U}$ is denoted as $\mathcal{X} \times_n \mathbf{U}$ and results in a tensor of dimensions $I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N$. Element-wise, the n -mode product is defined as:

$$(\mathcal{X} \times_n \mathbf{U})_{i_1, \dots, i_{n-1}, j, i_{n+1}, \dots, i_N} = \sum_{i_n=1}^{I_n} x_{i_1, \dots, i_n} \cdot u_{j, i_n} \quad (6)$$

The n -mode product can be thought of as multiplying each slice of the tensor along the n -th mode by the matrix $\mathbf{U}$. [7, p. 460]

Finally, one also needs to define the concept of rank for tensors, which is a generalization of the concept of rank for matrices. The rank of a tensor is defined as the minimum number of rank-one tensors that are needed to express the tensor as a sum. A rank-one tensor is a tensor that can be expressed as an outer product of vectors. Mathematically a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ has rank R if it can be expressed as a sum of R rank-one tensors, but not fewer. So,

$$\mathcal{X} = \mathcal{X}_1 + \mathcal{X}_2 + \dots + \mathcal{X}_R \quad (7)$$

where each $\mathcal{X}_r$ is a rank-one tensor. Unlike the matrix case, determining the rank of a tensor is generally NP-hard, which means that there is no known efficient algorithm to compute the rank of a tensor in general. Moreover, the rank of a tensor can be much larger than the dimensions of its modes. [7, p. 465]

Decomposition methods

Now that a general introduction to tensors has been given, one might consider how this data can be decomposed, like PCA or SVD for matrices. This is where tensor decomposition methods come into play. A problem with tensors arises due to the sheer amount of data needed to be stored. Therefore, tensor decomposition methods are used to reduce the dimensionality of the data while preserving its structure and information, as is the focus in this paper.

There are several different tensor decomposition methods, each with its own advantages and disadvantages. The difference in decomposition methods for tensors against normal matrices is that there are multiple ways to decompose a tensor. [7, p. 462]

CP/Parafac

Firstly, the CP/Parafac decomposition is introduced. It is based on the idea of expressing a tensor as a sum of rank-one tensors. For example, given a third-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$, the CP decomposition can be expressed as:

$$\mathcal{X} = \sum_{r=1}^R \lambda_r \mathbf{a}_r \otimes \mathbf{b}_r \otimes \mathbf{c}_r \quad (8)$$

where λ_r are the weights, $\mathbf{a}_r \in \mathbb{R}^{I_1}$, $\mathbf{b}_r \in \mathbb{R}^{I_2}$, and $\mathbf{c}_r \in \mathbb{R}^{I_3}$ are the factor vectors, and R is the rank of the decomposition. The CP decomposition is useful for identifying the underlying structure

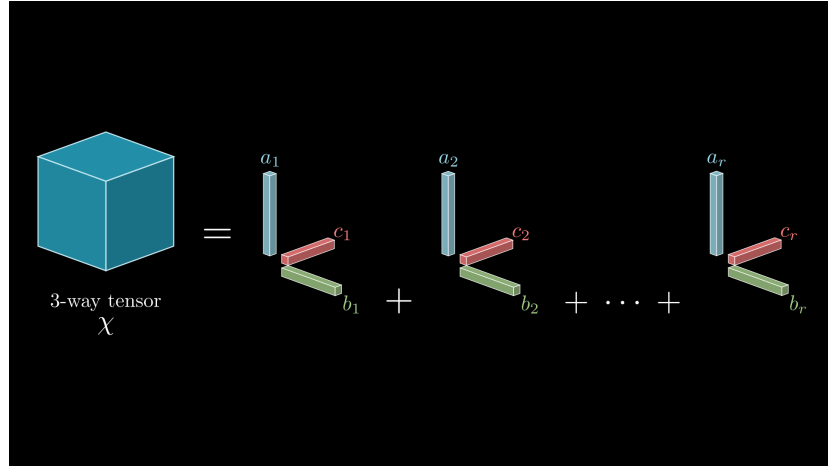


Fig. 1: Illustration of CP decomposition.

of the data and can be used for tasks such as dimensionality reduction, feature extraction, and data compression. [7, p. 463]

When it comes to computing the CP decomposition, as there are no finite algorithms that can find the rank of a tensor, the CP decomposition is found with iterative algorithms that approximate the decomposition. In practice, the CP decomposition is typically computed using the Alternating Least Squares (ALS) algorithm, where all factor matrices except one are fixed, and the remaining one is updated by solving a linear least-squares problem. This process is repeated until convergence. [7, p. 471]

Tucker

The Tucker decomposition is another popular tensor decomposition method that is based on the idea of expressing a tensor as a core tensor multiplied by factor matrices along each mode. The Tucker decomposition is thought of as a higher-order generalization of principal component analysis (PCA) for matrices. For example, given a third-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$, the Tucker decomposition can be expressed as:

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C} \quad (9)$$

where $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times R_3}$ is the core tensor, $\mathbf{A} \in \mathbb{R}^{I_1 \times R_1}$, $\mathbf{B} \in \mathbb{R}^{I_2 \times R_2}$, and $\mathbf{C} \in \mathbb{R}^{I_3 \times R_3}$ are the factor matrices, and R_1 , R_2 , and R_3 are the ranks of the decomposition along each mode. The factor matrices work as the principal components of the data along each mode, while the core tensor captures the interactions between the different modes. Like the CP decomposition, the Tucker decomposition is also commonly computed using an ALS-like algorithm (TUCKALS3, HOOI). [7, p. 477]

Tensor Train (TT)

The Tensor Train (TT) decomposition, also known as Matrix Product State (MPS) in quantum physics, as described in [9], is a tensor decomposition method that expresses a higher-order tensor as a chain of third-order core tensors. The TT decomposition is particularly useful for very high-order tensors, where methods like Tucker become impractical due to the exponential growth in the number of parameters.

Given a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the TT decomposition writes each element as:

$$\mathcal{X}(i_1, i_2, \dots, i_N) = \mathbf{G}_1[i_1] \cdot \mathbf{G}_2[i_2] \cdots \mathbf{G}_N[i_N] \quad (10)$$

where each $\mathbf{G}_k[i_k]$ is an $R_k \times R_{k+1}$ matrix, obtained by fixing the index i_k in the third-order core tensor $\mathcal{G}_k \in \mathbb{R}^{R_k \times I_k \times R_{k+1}}$. The boundary conditions require $R_1 = R_{N+1} = 1$, so that the chain of matrix multiplications yields a scalar. The values $R_2, R_3, \dots, R_N$ are called the TT-ranks. [10, p. 9]

The key advantage of the TT decomposition is its storage complexity. A full tensor with N modes, each of size d , has d^N elements, which grows exponentially in N . The TT representation on the other hand is less data intensive. If each core matrix $\mathbf{G}_k[i_k]$ is of size $r \times r$ for all k , the total number of parameters becomes Ndr^2 , which is linear in N for a constant r . The point of the method is to choose r such that a good and compact approximation of the original tensor is obtained. This makes TT well suited for situations where the tensor order is large but the data has low-rank structure, as it avoids the exponential growth in parameters that the Tucker decomposition suffers from when the tensor order increases. [11, p. 2] Furthermore, the TT decomposition can be computed via a sequence of SVDs (TT-SVD). [9, p. 2301]

Comparison of decomposition methods

	CP	Tucker	TT
Parameters	$R \sum_n I_n$	$\prod_n R_n + \sum_n I_n R_n$	$\sum_n R_n I_n R_{n+1}$
Scaling in N	Linear	Exponential	Linear
Uniqueness	Yes (mild conditions)	No	No
Algorithm	ALS	ALS	SVD sequence

Table 1: Comparison of CP, Tucker, and TT decompositions for an N th-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$.

[7, p. 468, 479]

Tensor diagrams

Tensor network diagrams are a graphical notation commonly used to visualize tensor decompositions [10, p. 9]. In this notation, each tensor is represented as a node (circle), and each mode of the tensor is represented as an edge (line) extending from the node. A shared edge between two nodes represents a summation over that index. A free edge represents a mode of the original tensor that is not summed over. For example, a vector has one free edge, a matrix has two, and a third-order tensor has three.

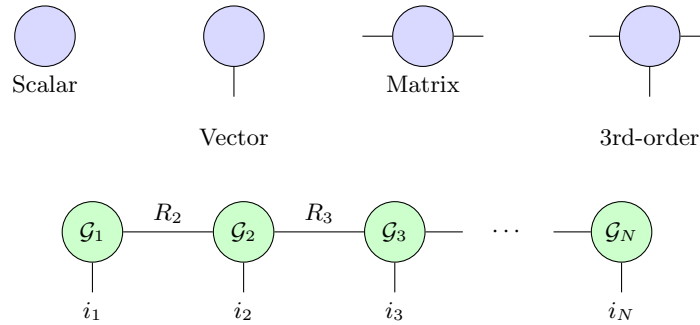


Fig. 2: Top: tensor diagram notation for tensors of different orders. Each edge represents a mode. Bottom: the Tensor Train decomposition represented as a tensor diagram. The vertical edges are the physical indices (modes of the original tensor), and the horizontal edges are the virtual bonds with TT-ranks R_k .

Reinforcement learning

Reinforcement learning is fundamentally based on the framework of a **Markov Decision Process (MDP)**, which provides the mathematical structure for sequential decision-making under uncertainty. An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the set of possible states, $\mathcal{A}$ is the set of possible actions, $P(s_{k+1} | s_k, a_k)$ is the transition probability distribution describing how the environment evolves in response to the agent's actions, $r(s_k, a_k, s_{k+1})$ is the scalar reward received after each

transition, and $\gamma \in [0, 1)$ is a discount factor that controls the weight given to future rewards. The Markov property – the assumption that the next state depends only on the current state and action, and not on any earlier history – is what makes the framework easily managed, and it is the foundation on which Q-learning is built.

The goal of the agent is to find a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ a mapping from states to actions that maximises the expected discounted return. The value of taking action a in state s under policy π is captured by the **Q-function**, also called the action-value function:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_k \mid s_0 = s, a_0 = a, \pi \right] \quad (11)$$

The optimal Q-function Q^* satisfies the **Bellman optimality equation**:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (12)$$

This recursive relationship is the theoretical basis for the Q-learning update rule introduced below – the update rule is essentially an iterative approximation of this equation, nudging the current estimate of Q towards the Bellman target at each step.

Tabular Q-learning

In tabular Q-learning, the agent moves in discrete timesteps from one state s to another state s' by choosing an action a in that state, and then receives a reward r . To keep track of how promising the different possible actions for a given state are, you store a list of expected future cumulative rewards for each action. This gives you one big Q-table, with all the possible states along one axis, and the values for the different actions along the other axis. As illustrated in figure 5, this Q-table can also be represented in a tensor with the same number of dimensions as our state space, which becomes important for our decomposition approach.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where α is the learning rate, and the discount factor γ control how much we weight the expected future reward from the next state and beyond.

The policy is given as choosing the action with the biggest action value for each state. The idea is that when the agent gets to a new state, it can look up the table to see which action it should take.

However since the agent has not played the game yet in the beginning, these values are first initialized either randomly or with the same low value. From here on, it will one by one update the Q-table values by performing different actions, observing what rewards it receives, and updates the Q-table accordingly. Since the agent will always pick the most promising action, we make it perform a random action with probability ϵ while training, in order to make it explore other actions as well. This ϵ is typically made smaller and smaller throughout the training session.

This Q-table can become quite large for environments with big state-spaces because you have to store the action values for every single state. This is the reason we, throughout this report, have experimented with approximating it with different amounts of base tensors.

Deep Q-learning

In deep Q-learning, the agent uses a neural network to approximate the Q-function instead of a Q-table. The neural network takes the state as input and outputs the action-values. The training process is similar to tabular Q-learning, however instead of the classical Q-learning update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (13)$$

We use a loss function to train the neural network.

The network is trained by minimising the **mean squared error** between the predicted Q-value and the Bellman target:

$$\mathcal{L}(\theta) = \left(y - \hat{Q}(s, a; \theta) \right)^2 \quad (14)$$

where y is the Bellman target, and θ denotes the parameters of the target network, held fixed during the update.

Furthermore, in deep Q-learning, a number of techniques are used to stabilize the training process, as this can be a difficult challenge during training. These techniques include elements such as experience replay, target networks, and double Q-learning. Experience replay involves storing the agent's experiences in a replay buffer and sampling mini-batches of experiences to train the neural network. Target networks are used to stabilize the training process by using a separate network to calculate the target values for the Q-learning update. This works by keeping a copy of the neural network, and fixing the parameters of this copy for a certain number of training steps, and then updating it to match the current neural network. [12, p. 462] Double Q-learning is a technique that helps to reduce overestimation bias in the Q-values by using two separate neural networks to estimate the action-values. The idea is to use one network to select the action and the other network to evaluate.

For deep Q learning, the state space can be very large, and the neural network can be used to approximate the Q-function without having to store a Q-table. However, this can still be computationally expensive, and this is where tensor decomposition methods, such as CP, Tucker and Tensor Train can come into play to reduce the dimensionality of the data while preserving its structure and information. Since, DQN is about applying deep learning to approximate a Q function, it is natural to also transfer the tensor decomposition methods that might be used in other domains in deep learning, to the domain of reinforcement learning. The following sections explore how this is realised in practice.

Method: Experimental Setup

We evaluate the effectiveness of tensorized representations in reinforcement learning across both tabular and function-approximation settings. Our experiments are designed to assess (i) learning performance, (ii) sample efficiency, and (iii) parameter efficiency of tensorized methods compared to standard baselines.

Overview of experimental approach

As we want to investigate tensor networks in tabular q-learning and deep q-learning, the experimental section of the project is divided into two phases. The first phase focuses on implementing a tabular Q-learning algorithm and exploring how tensor decompositions can improve the efficiency of the model. The second phase involves implementing a deep Q-learning algorithm and applying various methods to enhance its performance. Finally, the second phase also includes a deep dive into comparing the performance of a standard deep Q-network (DQN) against a tensor network-based matrix product state (MPS) learning approach across multiple maps in the Gymnasium environment.

Environments

Since reinforcement learning is based on interacting with an environment rather than a dataset in the classical sense, the environment still acts as the data component of the experiment. This is especially true since we pre-sampled a large pool of random state-action pairs from the environment

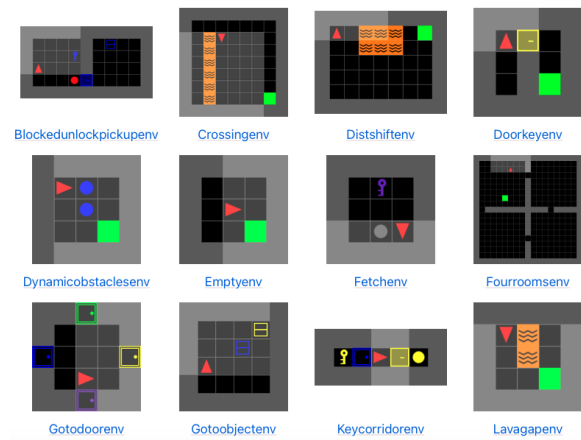


Fig. 3: Some of the pre-made levels provided by the Minigrid library

for kickstarting the training of the reinforcement learning model before effectively deploying it in the environment.

For our environment we chose the Gymnasium Minigrid environment. This is a fully OpenAI Gym-compatible grid-based environment maintained by the Farama organization. It consist of a 2-dimensional grid where an agent has to reach a goal position. Depending on the configuration, multiple sub-goals can be added where the agent for instance has to pick up keys to enter through doors, push around objects or avoid lava-tiles. The agent can only see a certain amount of tiles near its position. All of these variables add dimensions to the tensor representing the game state, which is the kind of problem where tensor decompositions have the best prerequisites to make a difference.

An important benefit of the Mini-Grid is that apart from the wide selection of pre-made levels, it also provides source code that makes it very easy to modify or make your own levels, which allowed us to tweak the level to our specific needs.

Data generation

Before even deploying the agent in the environment and let it begin to take actions, we pre-sample 10-100 thousand random state-action pairs depending on map size to kickstart model learning and already most importantly, give it experience from a wide variety of states so that it starts with the state space better explored than a fresh model would. This is because when models normally explore by playing the game and choosing actions themselves, they only get to see new states adjacent to currently explored ones.

Data buffer

When training by acting in an environment, consecutive transitions tend to be highly correlated. With these points being so similar, training on all of them would be unnecessary. In addition, a replay buffer allows samples to be stored and re-used multiple times similar to epochs in classical supervised machine learning, making it possible to generate fewer data points to train on. Therefore, we cache the last 100 thousand transitions and only pick a random few to actually train on, in our case 64 samples.

Environment: MiniGrid-Empty

MiniGrid-Empty is a grid-world environment, with discrete observation and action space. Like the name implies, the map is there are no obstacles or keys it has to collect, the agent simply has to reach the terminal state in the bottom right corner. The agent can either have a fixed or random starting position. This simple setting allows for a good baseline environment, and it is easy to track where the agent might fail. The MiniGrid-Empty environment has 4 actions (Up, Down, Left, Right), a living

cost of -1, and a terminal reward of 0. For a 5x5 grid, the optimal cost is -7.

Environment: MiniGrid-DoorKey and MiniGrid-DistShift

Apart from the default empty map, we also used two variations of the MiniGrid environment, which introduces a nudge of complexity to the environment. In the DoorKey-5x5 environment, the agent has to pick up a key and open a door before reaching the goal. [4] The environment has action space of 7, observation space of (7,7,3). The DistShift-5x5 environment is a variation of the empty map, where the agent has to avoid a lava pool before it can reach the goal. This environment has an action space of 7, and an observation space of (7,7,3). [3]

These environments were primarily used in the Phase 2: Deep Q-learning part of the project.

Environment: Custom 8D and 10D

In order to aid the testing process, and the understanding the impact of spatial structures in these models, two simple custom environments were created. The first one is a 10-dimensional navigation environment with 3^{10} state space, 30 observation dimensions and 20 actions. The second one is a simple 8-dimensional navigation environment with 3^8 state space, 24 observation dimensions and 16 actions. These environments are designed to be simple and easy to understand, while still providing a good testbed for the models. They were primarily used in the Phase 2: Deep Q-learning part of the project.

Environment: Minigrid-variations

Since the empty version of minigrid just requires the agent to move along the same approximate direction (i.e. diagonally to the bottom right corner), we made some additional variants with walls forcing the agent around different paths, sometimes going in the complete opposite direction of the goal. Since the CP model adjusts both the entire row and column fields in the same direction when learning from a state, making it go right in one end of a column, but left in the other, should challenge its capabilities. Adding maps with a lot of diagonal movement required should also contest the CP model, since it learns quicker horizontally and vertically. Some layouts have specific chicanes or reverse tracts that the agent must enter, and we expect the CP model to struggle at these specific break-points.

The layout variant "narrow" is almost identical to the empty layout, except the agent has to do a long, linear detour at the end. The "chicane" layout forces the agent to do a double deviation compared to the direct direction to the goal. The "islands" variant is designed as the most natural layout type having some random spots of walls scattered over the map. The "swirl" map is designed to be the most challenging variant to the model by making the agent go almost the completely opposite direction of the goal in both directions at the same time, before clearing a narrow tract to enter the goal.

In order to test how the CP model scales compared to the tabular model, we created each of these maps in an 8x8, 12x12, 16x16, 24x24, 32x32, and 48x48 grid size.

These variations can be seen in figure 4.

Fixed start vs. random start

One of the strengths with CP updating across a whole column and row is that it might learn unseen regions much faster, assuming the overall movement direction should be the same across rows and columns. This is because when encoding direction for one field when learning, it also encodes some of this direction in all of the fields of the same row and column. Therefore, CP should learn much quicker on simple maps where the agent spawns a random place, instead of in the same corner. In comparison, when spawning randomly, the tabular method will have to learn the new path to the goal from each new region every time from scratch, which should take more time.

In order to test this hypothesis, we ran all map combinations with both a fixed spawn position in the upper right corner, and a random spawn position anywhere on the map.

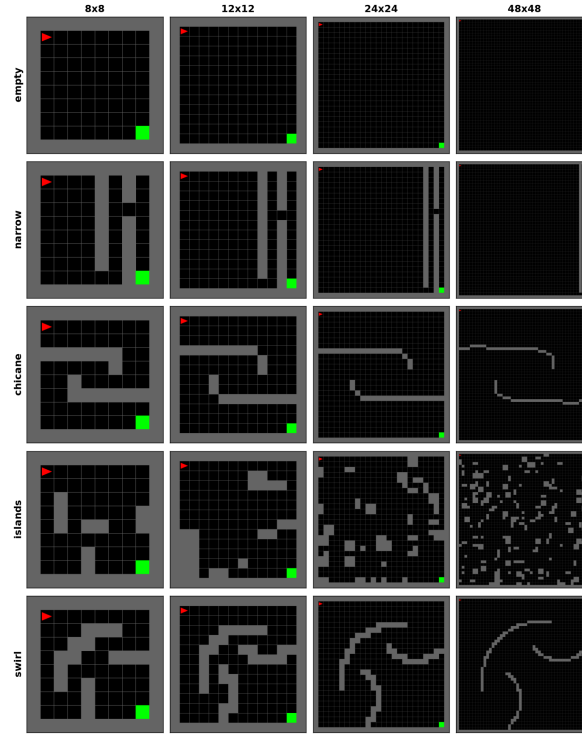


Fig. 4: Different types of maps with some of the different grid sizes used in testing.

Experimental Protocol

To ensure a fair comparison between the different Q-learning methods, all experiments are conducted using a common experimental protocol. Each agent is trained for a fixed number of environment steps using the same exploration strategy based on an ϵ -greedy policy. To account for the stochastic nature of reinforcement learning, every experiment is repeated with multiple random seeds, and the reported results are presented as the mean together with the corresponding standard deviation. During evaluation, exploration is disabled, and the learned policy is assessed over several evaluation episodes.

Evaluation Metrics

The performance of each method is evaluated using the following metrics:

- **Return:** The average episodic return over the final 20 evaluation episodes.
- **Value error:** The relative error between the learned Q-function, $\hat{Q}$, and the optimal Q-function, Q^* , computed using value iteration,

$$\frac{\|\hat{Q} - Q^*\|}{\|Q^*\|},$$

where $\|\cdot\|$ denotes the Euclidean norm.

- **Accuracy:** The percentage of states with the same greedy action as Q^* .
- **Parameter efficiency:** The achieved performance relative to the number of trainable parameters.
- **Sample efficiency:** The learning speed measured by the return as a function of the number of training steps.

Libraries and code structure

In order to implement the methods described above, several libraries are necessary. These libraries provide the necessary tools and functions to perform tensor operations and decompositions efficiently, as well as handling the game data and the training of the models.

Code structure

The main code structure of the project is organized as follows (see the GitHub repository):

- **code/**: Testing and unit implementation of the methods for RL and tensor decompositions, as well as the relevant files from Introduction to Reinforcement Learning course.
- **data/**: Sources and relevant data for the project, such as cited papers.
- **figures/**: Figures and visualizations used in the report.
- **log/**: Logs of AI usage and dialogues.
- **sim/**: Main implementation of the project, including the training loop, environments, models and tensor decompositions.

The core idea behind the `sim/` code was a modular structure, where the different components of the RL pipeline are separated into different files and classes. Thereby, when running code, one can easily swap out different components, such as the model architecture or the environment.

Gymnasium

The Gymnasium library is an open-source library for reinforcement learning environments and development. It provides a variety of pre-designed environments for usage in reinforcement learning settings. In this project, the Gymnasium library serves a fundamental role in providing the environment for the agent. However, custom environments were also created by modifying known GridWorld environments.

Torch

The PyTorch library which was implemented in this project is an open-source machine learning library. It is responsible for providing the main fundament behind the whole running and training of the models. More specifically, this project applied the PyTorch library to implement artificial neural networks in the parts of reinforcement learning where relevant.

Tensorly

Tensorly is a Python library for tensor learning. It provides an interface to manage and perform tensor operations and decompositions. It supports various backend systems as numpy, PyTorch, TensorFlow and SciPy, which makes integration of tensors into model training easier. [1] In practice, this project imports the Tensorly library and uses it in the training of the DQN.

Phase 1: Tabular Q-learning setup

In the tabular setting, we compare standard tabular q-learning with the same setting but a CP representation of the Q-table.

The evaluated methods are:

Table 2: Overview of the evaluated methods.

Method	Rank(s)	Environment Gridsizes
Tabular	–	8x8, 12x12, 16x16, 24x24, 32x32, 48x48
CP	3, 6, 12	8x8, 12x12, 16x16, 24x24, 32x32, 48x48

In the tensorized approaches, the Q-table is represented as a multi-dimensional tensor and approximated using a low-rank decomposition.

CP Q-table representation and update

The Q-table is not stored explicitly as a dense tensor. Instead, it is represented in a factorized form using low-rank factor matrices, one for each dimension of the state-action pair. The Q-value for a given state-action triplet (row, col, a) is computed as

$$Q(\text{row}, \text{col}, a) = \sum_{r=1}^R A_{\text{row}}[\text{row}, r] A_{\text{col}}[\text{col}, r] A_{\text{act}}[a, r],$$

where R is the chosen rank of the factorization.

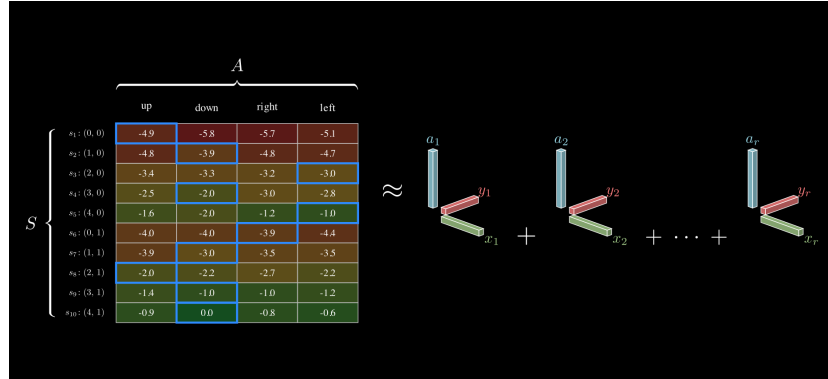


Fig. 5: The CP approximate representation of the Q-table, having an action, x , and y dimension. The blue outline on the Q-table marks the optimal policy.

Gradient computation The gradient with respect to each factor is obtained using a leave-one-out structure. For a given factor A_d and index i_d , the gradient is given by the element-wise product of the corresponding rows in the remaining factor matrices:

$$\frac{\partial Q}{\partial A_d[i_d]} = \prod_{d' \neq d} A_{d'}[i_{d'}],$$

where the result is a vector of length R . For example, for a state-action triplet (row, col, a), the gradients are

$$\begin{aligned} \frac{\partial Q}{\partial A_{\text{row}}[\text{row}]} &= A_{\text{col}}[\text{col}] \odot A_{\text{act}}[a], \\ \frac{\partial Q}{\partial A_{\text{col}}[\text{col}]} &= A_{\text{row}}[\text{row}] \odot A_{\text{act}}[a], \\ \frac{\partial Q}{\partial A_{\text{act}}[a]} &= A_{\text{row}}[\text{row}] \odot A_{\text{col}}[\text{col}], \end{aligned}$$

where $\odot$ denotes element-wise multiplication.

NLMS update rule The parameters are updated using a normalized least-mean-squares (NLMS) rule. Let the temporal-difference error be

$$\delta = \text{clip}(r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a)).$$

The update is then given by

$$A_d[i_d] \leftarrow A_d[i_d] + \frac{\eta \delta}{\|g\|^2 + \epsilon} g_d,$$

where g_d is the gradient for factor d , η is the learning rate, and $\|g\|^2$ is the sum of squared norms of all factor gradients.

The normalization by $\|g\|^2$ ensures that the effective update size of $Q(s, a)$ is approximately proportional to $\eta\delta$, independent of the rank R . This prevents gradient explosion for higher-rank factorizations and allows the use of relatively large learning rates.

TD target with target network The temporal-difference target is computed using a frozen copy of the factor matrices:

$$y = r + \gamma \max_{a'} Q_{\text{target}}(s', a').$$

The target Q-function is evaluated using parameters that are periodically synchronized with the online network. This stabilizes learning by preventing the bootstrap target from changing at every update step.

Computational efficiency

A key advantage of the CP representation is its computational and memory efficiency. At each update step, only the factor rows corresponding to the current state-action tuple are modified, namely $A_{\text{row}}[\text{row}]$, $A_{\text{col}}[\text{col}]$, and $A_{\text{act}}[a]$. All other parameters remain unchanged, resulting in an update whose computational complexity is linear in the decomposition rank R rather than in the size of the full Q-table. The memory requirements are also significantly reduced. For tabular Q-learning, the number of parameters equals the size of the state-action space,

$$\text{Parameters} = |\mathcal{S}| |\mathcal{A}|.$$

For an empty GridWorld with dimensions $x_n \times y_n$, this becomes

$$\text{Parameters} = x_n y_n |\mathcal{A}|.$$

In contrast, a CP decomposition requires only

$$\text{Parameters} = R(x_n + y_n + |\mathcal{A}|),$$

where R is the decomposition rank. Consequently, the number of parameters grows linearly with the dimensions of the state and action spaces, rather than multiplicatively as in the tabular representation. This reduction in memory becomes increasingly significant as the environment size grows, as illustrated in Figure 6.

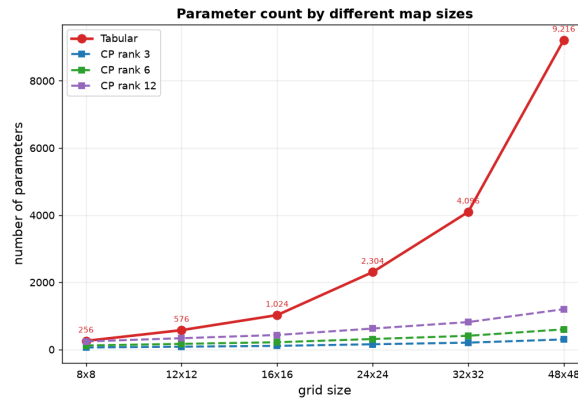


Fig. 6: Growth in the number of parameters for the tabular and CP representations as the GridWorld size increases.

Phase 2: Deep Q-Learning setup

In the second phase, we implemented a variety of methods to improve the performance of the deep Q-learning algorithm, as introduced in the earlier in the method section. This process took many resources and underwent much trial and error; which is why there are multiple *versions* of data collected, through different versions and approaches to the modeling of the structures into the DQN code. These different versions with different environments and architectures can be found in the code repository, and are categorized as follows:

- *cases/*: Version 1, with 6 instances. This includes the first working iteration of the DQN code being run with benchmarks on the different maps. As there were some fundamental issues with the structure of our models, this version is more used data / evidence for alternative perspectives.
- *designs/ from design 0 to design 4* Version 2, with 5 instances. The main data collected for the Phase 2 on deep Q-learning, with the different cases of tensorization and decomposition methods implemented, and tested on the different maps.
- *designs/ from design 5 to design 6* Version 2.5, with 2 instances, extra data collected in order to research the impact of longer training runs and more spatially complex / larger maps.

The versions implemented amount to around 6 *cases* of different ways to integrate tensor and tensor decompositions into the deep Q-learning algorithm.

designs/ explained

The different designs implemented in phase 2 include:

- Design 0: Baseline DQN
- Design 1: CP Q-function
- Design 2: Neural Network Weight Decomposition
- Design 3: Tensorized Input
- Design 4: MPS Tensor Network

Now going through the different designs.

The first design, design 0, is the baseline DQN, which is an implementation of the DQN algorithm without any tensorization or decomposition. This serves as a control for comparison against the other designs. In order to improve stability and performance, this is a double DQN, which is a common improvement to the standard DQN algorithm that helps to reduce overestimation bias in action value estimates. As mentioned in the theoretical method section, the double DQN algorithm uses two separate neural networks to estimate the action values, which helps to mitigate the overestimation bias that can occur in standard DQN. The network has two hidden layers of size 128, and uses ReLU activation functions. In terms of configuration, learning rate is set to 0.001, discount factor γ is set to 0.99, and the exploration strategy is ϵ -greedy with ϵ decaying from 1.0 to 0.05 over the course of training. In terms of parameters, this model has approximately 36,359 parameters, in the given minigrid environment.

The second design, design 1, is the CP Q-function which represents the Q-function as a CP (CAN-DECOMP/PARAFAC) tensor. In this design, the entire value function is represented as a sum of rank-1 tensors, where each factor function is a lookup table or small linear map over the corresponding dimension of the state and action space. This design does not use a neural network, and there are no activation functions or weight matrices being decomposed. Instead, the Q-function itself is directly represented as a CP sum, and the factor vectors are learned directly. This can be shown mathematically as: [11, p. 6]

$$Q(s_1, s_2, \dots, s_n, a) = \sum_r u_r(s_1) \cdot v_r(s_2) \cdot \dots \cdot z_r(a) \quad (15)$$

The rank of the CP decomposition can be varied to control the number of parameters in the model, and we evaluate the performance of different ranks to understand the trade-off between model complexity and performance. In the formula, the rank R determines the number of rank-1 terms in the CP decomposition.

The parameter size of this model can be calculated with the formula $R \cdot (7 + 7 + 3 + 7)$, where R is the rank of the decomposition, and given the input size of the MiniGrid observation (7,7,3) and the output size of the action space (7).

This design is also the one used in the Phase 1: Tabular Q-learning setup.

For the next design, design 2, we implemented a neural network weight decomposition, where the weight matrices of the neural network are replaced or approximated by tensor decompositions. The network still computes $y = Wx + b$ (same function), but W is parameterized differently. There are two sub-cases of this design: per-layer weight factorization and tensorized layer (multilinear operations). In the per-layer weight factorization, each fully connected layer is replaced with a factorized equivalent using either CP or Tucker factorization. In the tensorized layer approach, the input to each layer is kept as a tensor and the layer computes multilinear operations on the input using factor matrices. The expected difference in outcome between these two sub-cases is that the per-layer weight factorization compresses the weight matrix while still using a flat input, whereas the tensorized layer approach operates on tensor-structured input directly without flattening, which can potentially exploit the multi-dimensional structure of the data more effectively. [10, p. 12] Mathematically expressed, the per-layer weight factorization can be represented as:

$$W \approx \mathbf{F}_{\text{out}} \cdot \mathbf{F}_{\text{in}}^T \quad (16)$$

for CP factorization, and

$$W \approx \mathbf{U}_{\text{out}} \cdot \mathbf{G} \cdot \mathbf{U}_{\text{in}}^T \quad (17)$$

for Tucker factorization. The tensorized layer approach can be represented as:

$$Y = X \times_1 A \times_2 B \times_3 C \quad (18)$$

where X is the tensor input, and A, B, C are the factor matrices.

The parameters for these models are based on the configured rank size. However for rank-4 settings the parameter size can be expected to be around 2000-5000 parameters.

Design 3 is the tensorized input design, where the input state is reshaped into a tensor to exploit potential multi-dimensional structure. In this design, the state vector is kept as / reshaped into a tensor and fed into a tensor-network embedding layer that maps this structured tensor to a compact hidden vector. The embedding can be done using either TT (MPS) or CP decomposition. After the embedding, the rest of the network is a standard MLP. The key point of this design is purely the input compression/structuring step, while the downstream network remains unchanged. Mathematically, the input state $s \in \mathbb{R}^d$ is reshaped into a tensor $S \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_n}$, and then passed through an embedding layer to produce a hidden vector.

The last design, design 4, is the MPS tensor network design, where the entire model is a tensor network without any dense layers. The MPS tensor network follows the theory introduced in the method: theoretical method section as tensor train (TT) decomposition. Thus, this design does not base itself on a neural network architecture where the layers are decomposed, but instead it works as its own model.

In this design, the state is factored into components. The implementation is based on the paper [11, p. 6], which describes how a classifier F can be represented as an MPS. By looking at F as the Q-function, we can use the same general idea of representing the parameters of the Q-function with decomposed tensors. In our case, the flat input state is split into n groups, and each group is passed through a small

linear layer (learned), that produces a feature vector for that group. These can be viewed as local data embeddings. The argument for this is that the flat input state must be reshaped into chunks that can be fed into the MPS. Using one MPS core per group, the n feature vectors are then multiplied through a chain of MPS cores: $\mathbf{A}^{(k)} \in \mathbb{R}^{r \times d_{\text{phys}} \times r}$ to compute the output of the model. From this, the final core output is used to determine the Q-values. The reason why MPS is different from design 1 with CP, is that the MPS cores interact locally through the bonds, allowing for more correlated features.

Other versions

Apart from the *designs/* 0-4, which are the main data collected for the phase 2 of the project, there are also the *cases/* versions, which are the first iterations of the DQN code, as well as the *designs/* 5 and 6 which are extra data collected.

The interesting data from the *cases/* might include observations from CNN implementation. A bit more detailed description of the *cases/* versions can be found appendix: *cases/* directory explained.

As mentioned earlier, the *designs/* 5 and 6 focus on running all the different designs for longer training runs with a larger map size.

Results

Phase 1: Tabular Q-learning Experiment

Accuracy

The overall accuracy metrics after training for the CP model compared to tabular Q-learning quickly splits into two different conclusions (figure 7). When the agent has a fixed start by spawning in the same upper left corner every time, CP performs substantially worse than tabular. However, when the agent spawns at a random spot, the CP model matches the performance of tabular.

For the fixed starting point, only at 8x8 grid size does CP rank 12 match the performance of tabular at close to 100%. However for all other grid sizes, the CP models perform worse at around 70% accuracy compared with about 90% for tabular. For these other grid sizes it also does not seem improve the accuracy with adding CP rank, pointing to a more general problem with the learning process when the starting point is fixed.

For the random starting point, the CP models seem to perform as well as the Q table, depending on rank. Rank 12 and 6 seems to match tabular perfectly well, with rank 12 sometimes scoring a tiny bit higher whilst rank 6 scores a bit lower, and rank 3 is only around 10% points from tabular.

Common to both the fixed and random start, the empty map gets fully learned by all configurations scoring at least 95% in final accuracy.

When looking at the different map variations (figure 14 in the appendix), "swirl" seems to be the hardest, followed by "narrow", "chicane" and then "islands". The pattern is the same for both fixed and random, however with fixed just being constantly worse.

Learning rate

CP methods seem to learn much quicker (figure 8). In all cases they almost instantly hit 75% accuracy, which they stay on for fixed start, whilst they improve about 10 more percentage points for the random start. All of this is while the tabular method slowly builds up accuracy.

Efficiency

When adjusting the accuracy metrics with the number of parameters for the different models, the CP models perform better than tabular (figure 9). That is, when reducing the number of parameters when going from tabular to CP, the accuracy score does not decrease as much relatively. Since number of parameters reflect memory usage, the CP methods has a better memory efficiency and will take up less RAM at runtime (however with the tests of our scale, the difference in RAM usage is negligible).

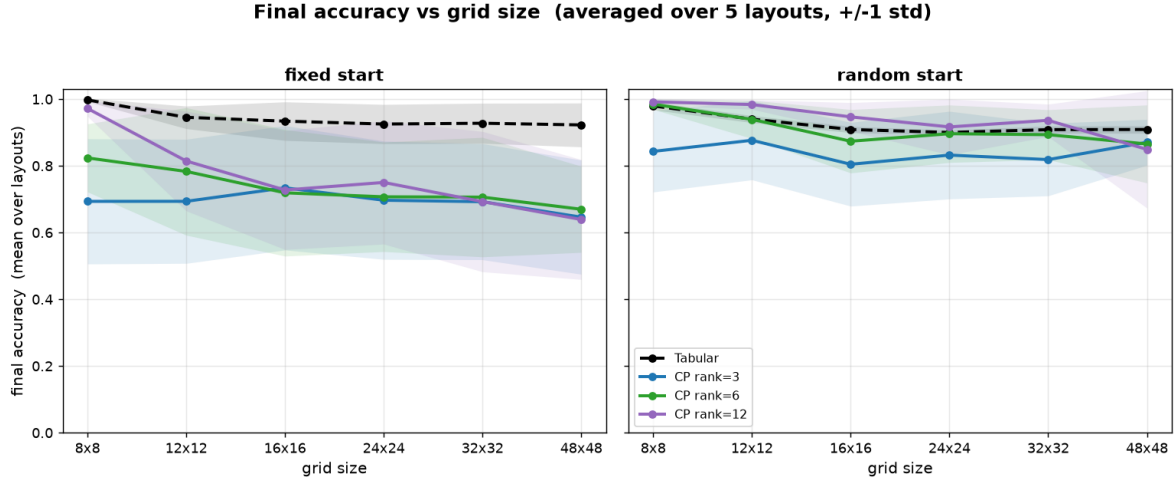


Fig. 7: Final training accuracy across grid sizes. This is averaged over different map layout.

Specifically, at the 8x8 maps, rank 3 CP scores about a 3-4x higher accuracy per 1000 parameters compared to tabular, with rank 6 scoring 2x and rank 12 performing about as efficiently. However as the grid size increases, this difference grows to 20-29x for rank 3, about 11-15x for rank 6 and 5-7x for rank 12. Therefore, larger grid size leads to an increasing memory efficiency gain for the CP methods.

This efficiency gain is however limited to memory usage, and does not carry over to compute time reduction. Even though we did not do a specific study or measurement of this, we did not experience any time reduction when doing a CP run vs. a tabular run. In fact CP should be a bit slower since it requires more compute steps both at inference but also training.

Phase 2: Deep Q-learning Experiment

The results of the deep Q-learning experiment are summarized in Table 4, which compares all designs across three environments of increasing difficulty.

Method	Best Config	Params	Empty	DoorKey	DistShift
Design 0: Baseline MLP	128 × 128	36,359	97%	6%	62%
Design 1: Pure CP	cp_r4 / r32	103-775	96%	28%	0%
Design 2A: CP weight decomp	cp_r8 / r16	5,591-10,919	99%	13%	57%
Design 2B: Tensorized layer	r2 / r16	2,097-529,687	96%	40%	78%
Design 3: TT embedding+MLP	tt_r2 / r16	18,353-25,591	99%	29%	78%
Design 4: FeatureMap MPS	r8_d16	13,815	29%	~0%	~0%

Table 3: Three environments for each design. Best-performing configuration per environment; ranges indicate variation.

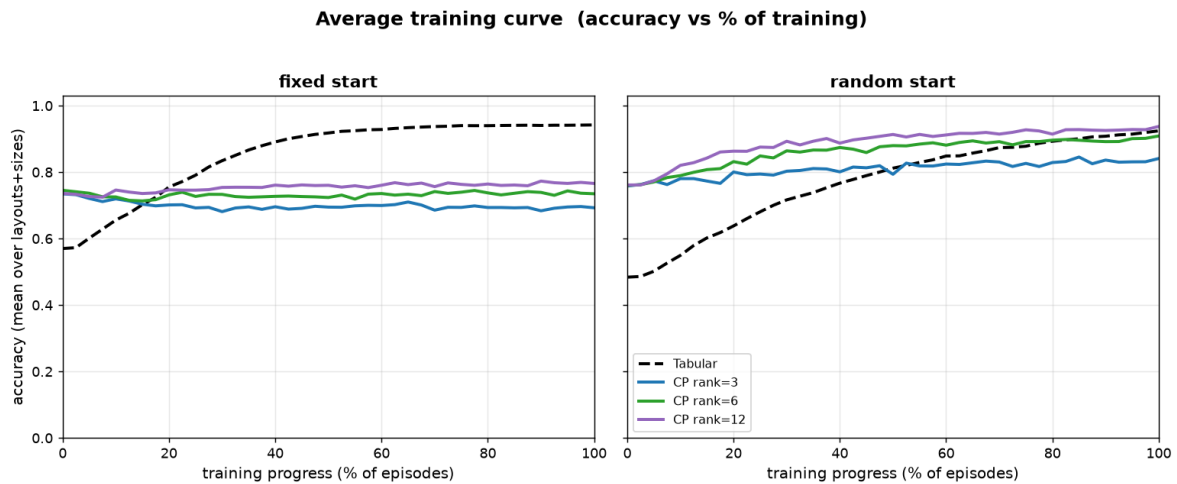


Fig. 8: Average accuracy during training across grid sizes and map layouts. Since bigger layouts have more training episodes, this metric has been standardized before averaging.

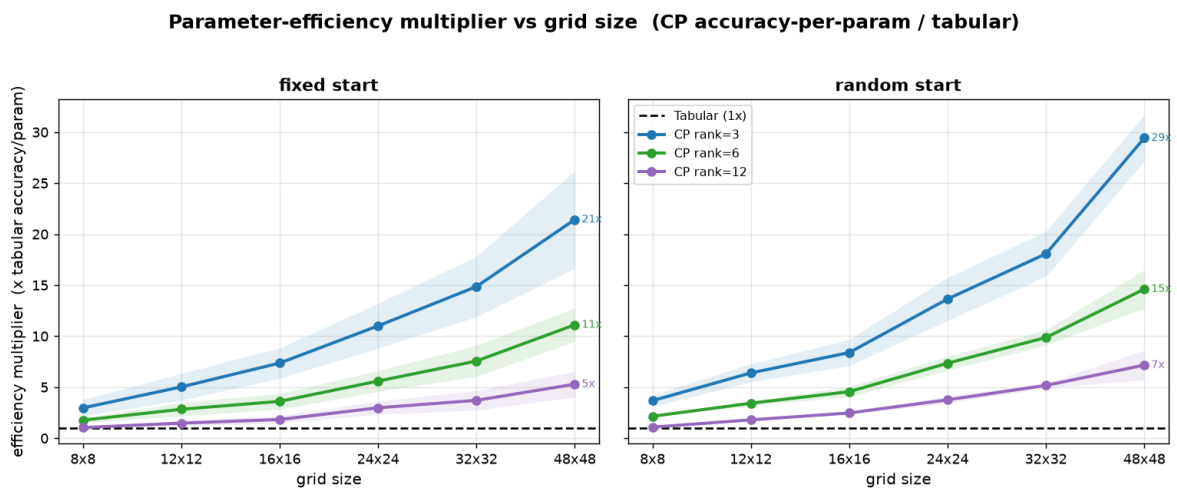


Fig. 9: Efficiency multiplier across grid sizes, calculated as accuracy score per 1k parameters.

Method	Best Config	Params	Empty	DoorKey	DistShift
Design 0: Baseline MLP	128 × 128	100%	97%	6%	62%
Design 1: Pure CP	cp_r4 / r32	0.3-2.1%	96%	28%	0%
Design 2A: CP weight decomp	cp_r8 / r16	15.3-30.0%	99%	13%	57%
Design 2B: Tensorized layer	r2 / r16	5.8-1,456.8%	96%	40%	78%
Design 3: TT embedding+MLP	tt_r2 / r16	50.4-70.4%	99%	29%	78%
Design 4: FeatureMap MPS	r8_d16	38.0%	29%	~0%	~0%

Table 4: Three environments for each design. Best-performing configuration per environment; ranges indicate variation.

Table 4 displays the best-performing configuration for each design across the three environments. The results show that the designs all perform well on the simplest environment (Empty-Random-5x5), with solve rates above 95%; shifting the perspective to a efficiency/compression-based one. As seen in the Phase 1: Tabular Q-learning experiment, the pure CP design (Design 1) achieves a high solve rate on environments with same-direction correlation in terms of the optimal route; so this is expected. As for the more complex environments, the results are more different. On DoorKey-5x5, the hardest environment, the best solving rate is only at 40%, achieved by Design 2B, while the baseline MLP only reaches 6%. On DistShift1, Designs 2B and 3 achieve the highest solve rates at 78%, while the pure CP design fails entirely. Design 4 (MPS) performs poorly across all environments, with solve rates around or below 30%.

Now focusing on the baseline, it can be observed that a double DQN with a (128,128) neural network MLP architecture performs very well on the simplest environment with 97% solve rate, but struggles significantly on the more complex environments, achieving only 6% on DoorKey and 62% on DistShift. Furthermore, from the first iteration of the data collection (*cases/*), where a similar baseline was trained denoted *Case 0a*, but with more training episodes (1000) and seeds (7), the solve rates on DoorKey remain below 20% (see appendix: Baselines across environments); and even when accounting for training volatility, the maximum solve rate observed during training was only 34% (see: Case 0a: DoorKey-5x5 1000 episodes output). This shows signs of fitting difficulties in between the baseline MLP and environment DoorKey. The performance of the baseline shows a performance ceiling on the DoorKey environment, which helps establish our reference environments as easy, intermediate and hard, respectively for Empty, DistShift and DoorKey.

When looking at the CP tensor decomposition design (Design 1), it can be observed that, when the policy is simple as for Empty, the CP demonstrates a high amount of efficiency at almost the same level of solve rate as the baseline MLP with 96% solve rate (see appendix: CP Q-function: learning curves (Empty-5x5)). The crucial point is however that only 103 params are needed for the CP model, while the MLP has 36,359 parameters; a 350x reduction (see appendix: CP Q-function: parameter efficiency (Empty-5x5)). When it comes to the other environments, the CP design struggles a bit more. For DistShift, the best solve rate is 0%, achieved with a CP rank of 4, which is a significant drop from the 62% solve rate of the baseline MLP. It can be argued that this is due to the CP design’s inability to store the required policy in its decomposed forms; like also covered in Phase 1: Tabular Q-learning. However on DoorKey, the best solve rate is 28%, achieved with a CP rank of 32 (see appendix: CP Q-function: learning curves (DoorKey and DistShift)), which is a large improvement from the 6% solve rate of the baseline MLP.

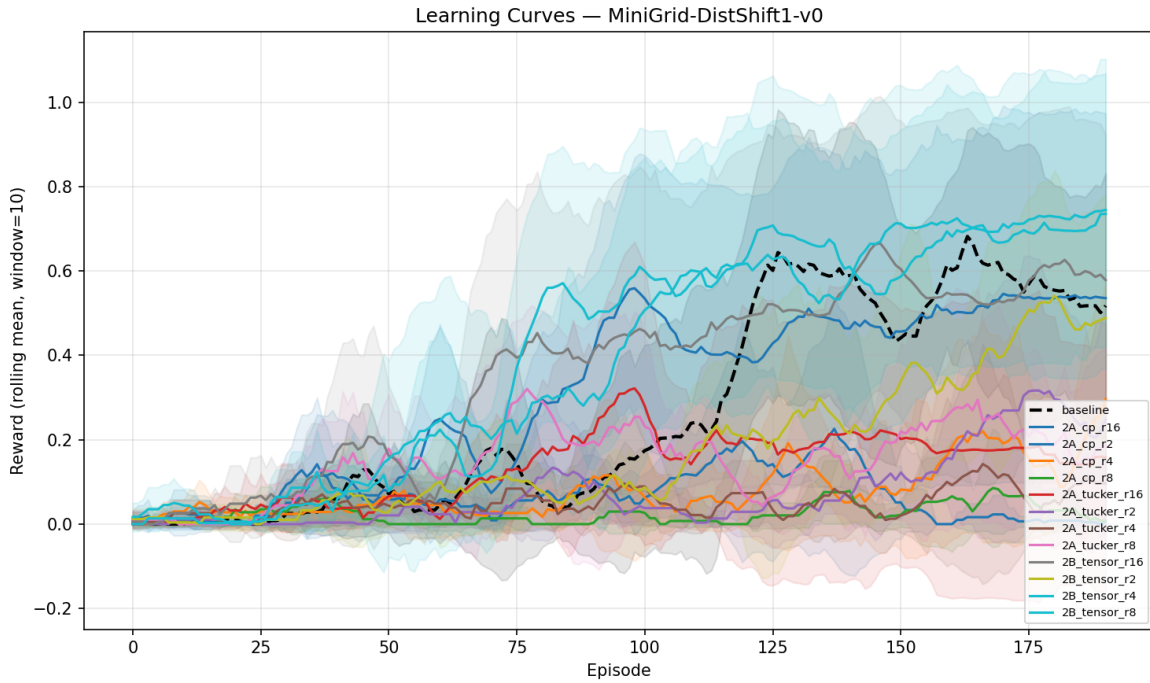


Fig. 10: Learning curves for Design 2 on DistShift1.

Interestingly, the CP weight decomposition design (Design 2A) perform quite well on the environments with 13% solve rate on DoorKey and 57% on DistShift, with only around 10,000 parameters needed for the best-performing Design 2a configuration: 2A_cp_r16 (see appendix: Weight decomposition: parameter efficiency (DistShift1)). When looking at the learning trend for this subdesign on DistShift (see appendix: Weight decomposition: convergence heatmap (DistShift1)), it can be observed that most Design 2A configurations do not learn to solve the environment. However this sparks the very interesting observation: that there is a clear performance gap between design 2a, which decomposes the weights per-layer, and the design 2b, which preserves the spatial structure using tensorized layers. Design 2b achieves a much higher solve rate of 78% on DistShift, and 40% on DoorKey, competing with all other model designs. The 40% solve rate on DoorKey being the highest solve rate among all designs. When looking at the learning curves for Design 2b on DistShift (see figure 10), it can be observed that many of the 2b configurations all learn to solve the environment. As can be seen in the parameter efficiency comparison for DistShift (see appendix: Weight decomposition: parameter efficiency (DistShift1)), the best-performing Design 2b configuration (2B_tensor_r4) achieves around 78% solve rate with only around 9,400 parameters; both exceeding the baseline in terms of solve rate and efficiency. Furthermore, a general insight is that all instances of Design 2b outperform Design 2a in training on the DoorKey and Empty environments (see appendix: Weight decomposition: parameter efficiency (DoorKey-5x5) and Weight decomposition: parameter efficiency (Empty-5x5)). This displays the importance of retaining and exploiting the spatial structure of the environments.

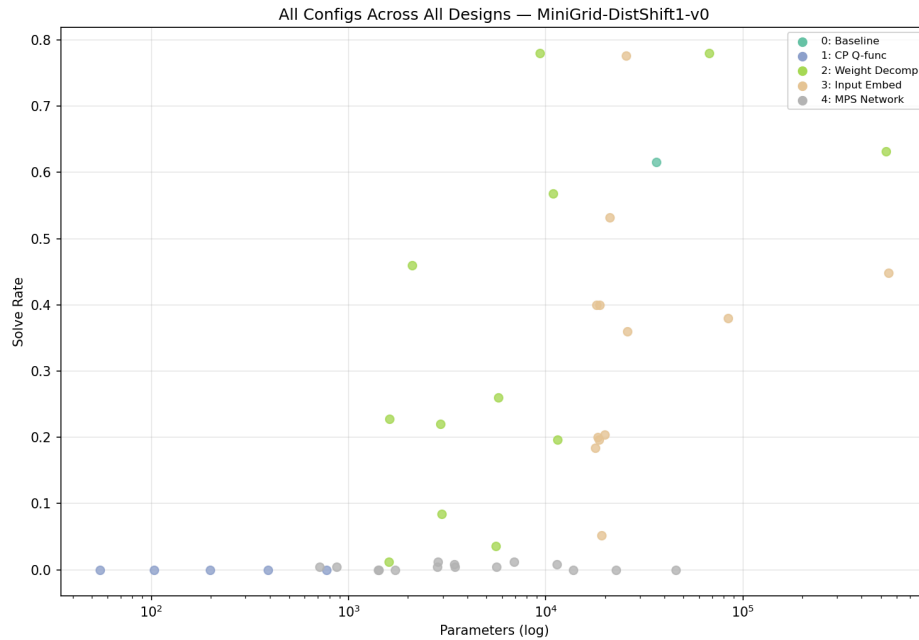


Fig. 11: Parameter efficiency comparison across all designs on DistShift1.

In terms of overall efficiency performance of Design 2b, it can also be seen on figure 11 where the weight decomposition is clearly competing for the most efficient model design on DistShift. This figure also shows that design 3 (input embedding) is also quite efficient on DistShift.

Design 3, which uses a tensorized input embedding layer followed by an MLP, performs quite well on all cases, yielding the second-highest solve rate on DistShift at 78% (25,591 parameters) and 29% on DoorKey. When looking at appendix: Input reduction: convergence heatmap (DistShift1), it can be observed that the config TT_r16 (tensor train rank 16) performs best. The convergence heatmap does not indicate clearly whether one decomposition method is better than the other.

Design 4, which is the MPS/TT-based architecture performs poorly across all environments. When looking at data, the model does not really learn anything over the course of training, with the only non-zero performance results being the 29% solve rate on Empty. However when looking at the appendix: MPS: convergence heatmap (Empty-5x5), it can be seen how the model actually fails at learning; it becomes worse over time, after its peak at 50% training. These results come despite the fact that the MPS designs reach up to +40,000 parameters.

Overall the results of the deep Q-learning experiment has shown that the best-performing models on the more complex environments (DoorKey and DistShift) are the ones that preserve the spatial structure of the environment, either through tensorized layers (Design 2b) or through input embedding (Design 3). The pure CP design (Design 1) performs well on the simplest environment, but struggles on the more complex environments; this is a parameter count vs performance trade-off. Additionally, the MPS-based design (Design 4) performs poorly across all environments. See appendix: Cross-design best solve rates. When it comes to best performing configs (ranks) of each model, generally speaking higher rank leads to better performance, this is the case for design1-doorkey, design2a-empty, design3-distshift and design3-doorkey. However, it is also observed that sometimes lower rank can match the performance or even outperform higher rank, e.g. design1-empty, design2b-distshift, design2b-empty. Interestingly, this is mostly the case for the more simple environments, where the policy is simpler and therefore intuitively does not require a high rank to be stored in decomposed form. Furthermore, one

can argue that some designs are more sensitive to rank than others, where for instance the Design 3 needing higher ranks to perform better.

At last, in order to research the impact of spatially larger environments and input embeddings, all designs were run on an 8D (named design 6 in the code) and 10D environment (named design 5 in the code), respectively. The main take-away from these experiments, is that tensorized Q-function as CP (design 1) actually performs quite well on these environments with 98% solve rate with rank 8. However, the other models did not attain any representable results (around zero), which might indicate that something went wrong in the training or implementation of these models; this can also be seen on all learning curves, which are all very volatile and unstable.

Discussion

General discussion on results and further improvements

Generally speaking, the research area can partly be seen as focusing on efficiency and optimizing the learning process, to reach the same amount of performance with fewer parameters – this is a compression based problem approach. Secondly, another perspective of the research is on improving the performance of the tensorized models, in terms of exploiting the structure of data in the environments, to reach more inductive bias in the models (e.g. retaining spatial structure leads to better solutions). This distinction is especially seen in the two phases of the project, where the first phase focuses mostly on how CP decompositions can be used to compress the Q-tensor / represent the Q-tensor. Thus, due to the inherent parameter reduction of CP decompositions and after reviewing the results, it can be argued that the CP decomposition mainly achieves the first point. Additionally, this is also seen in the training and research process, where the CP algorithm struggled with more complex environments; both in phase 1 and 2. For instance, the CP decomposition struggled with DoorKey and DistShift environments. Since, these environments are still relatively simple, and are still relatively constrained in terms of state-space size, it raises questions about the success of the decomposition methods in terms of the second point, as we would want to generalize to larger and more complex environments. On the other hand, it can also be argued that the CP decomposition might actually work well, but the learning process would require more careful tuning, as well as fitting to the specific environment - this is virtually the case in phase 1 of the project, where fine-tuning did improve the performance. In terms of the second point (on inductive bias), it can be argued that the Design 2b and Design 3 support it, as they both retain / exploit the spatial structure of the environments, and performed better than the other designs, and the baseline on some environments. Here, one could also argue that the baseline itself might not have been the best possible baseline; the choice of double DQN as baseline is discussed later. However, all things considered, given the same amount of training time, training steps and hyperparameters, the Design 2b and Design 3 did perform better.

Beyond the main designs, a CNN-based approach to tackle the spatiality of the environments was also tested in Phase 2, and showed promising results. This was inspired by the paper [10]. It was implemented as part of the *cases/* versions of the code (*cases/4*). From here, the CNN architecture which attaches a CNN onto the model before CP layers (backbone_cp) performed quite well on the MiniGrid Empty map, and could be further explored and optimized.

Some ideas that did not work well include the tensorized CNN, where the kernel filters were decomposed using tensor decompositions. This was also implemented as part of the *cases/4* version of the code. However, this did not perform well, which makes sense due to the limited amount of parameters in the CNN architecture, when comparing to other locations where one could apply decomposition. Furthermore, an approach of post-hoc compression was also tested (*cases/2b*). The idea was first to train a standard DQN model, and then apply CP decomposition to the learned weights of the model. This is the opposite order of the Design 2a, where the CP decomposition is applied before training. However, this post-hoc approach did not perform well; it could be explained by the fact that the CP

decomposition is not able to capture information of the learned weights correctly. After the training, it was observed that the agent kept performing the same action, thereby leading to 0% accuracy. This is however still an interesting idea, as it in theory should work just fine.

Where Tensor models shine

Throughout the experiments, it's evident where the tensor models shine – on the simple empty maps. This is seen in the result from phase 1 (Figure 14), as well as phase 2 (Table 4, with exception of Design 4), where they match or even outperform the baselines. There is an almost direct correlation between environment simplicity and relative tensor model performance, though this has not been quantified.

This strength is particularly visible in the convergence behavior. As seen in Figure 7, CP models in the random start setting also converge exceptionally fast, starting at a much higher accuracy compared to the tabular baseline, however not improving much over the episodes.

The reason for this lies in the structure of CP representation (Figure 1). In tabular Q-learning, updating a Q-value for a specific state-action pair affect only that single entry in the table – all other entries remain unchanged. In a CP model, however, each Q-value is computed as a product of rows from a shared factor matrices. Updating the Q-value for a given state-action pair therefore modifies the factor rows corresponding to that state and action, which in turn affects the Q-values of every other state-action pair that shares those factor rows – that is, every pair in the same row, column, or action dimension

This generalization effect has good and bad sides to it. In simple environments where the optimal policy has a consistent directional structure (right, down), this shared updating acts as a form of regularization, spreading useful information across the state space far quicker than tabular learning can. A single experience in one part of the grid immediately informs the model's estimates for spatially related states it has never visited. This is why CP converges so quickly on empty maps and why it benefits strongly from random start positions.

In more complex environments, however, this same property becomes a liability. When the optimal policy requires different actions in different parts of the same row or column – as in swirl (Figure 16) – the shared factor rows cannot represent these contradictory directions simultaneously. Updating on entry pulls the shared factors in a direction that degrades estimates elsewhere, and the model struggles to converge on a stable policy. This is the fundamental limitation of the CP decomposition, and it explains the performance gap between CP and tabular q-learning as environment complexity increases.

However, this limitation does not generalize equally to all tensor network architectures. The Tucker decomposition introduces a core tensor that captures interactions between modes, giving it more flexibility to represent policies that vary differently across rows and columns. Rather than sharing factor rows directly, Tucker controls the relationship between modes through this core, which means a single state-action update does not spread as strict across the entire factor structure. Tucker therefore trades some of the fast generalization that makes CP strong on simple maps for greater representational flexibility on complex ones – though at the cost of exponentially more parameters as the tensor order grows.

The Tensor Train decomposition occupies a middle ground. Like CP, it factorizes the tensor into a chain of smaller cores, but the sequential structure of the chain means that each core only directly interacts with its neighbors. Updates propagate locally along the chain rather than globally across all modes simultaneously, which gives TT more capacity to represent spatially varied policies than CP, while remaining far more parameter efficient than Tucker at high tensor orders. This is consistent with the Phase 2 results, where the TT-based designs – particularly Design 3 – outperformed the pure CP design on the more complex environments, suggesting that the local interaction structure of TT is better suited to tasks where the optimal policy cannot be compressed into a simple global factorization.

The deadly triad

During evaluation, we observed that the CP Q-tensor agent would initially learn a near-optimal policy, but afterwards collapse back to near-random performance. The CP representation itself was verified to be correct, as entry and gradient computations match analytical expectations, and gradient checks pass. The instability is therefore not a representational failure but a learning failure.

The root cause is the deadly triad [12]; the combination of function approximation, bootstrapping and off-policy learning. Because a single factor row is shared across many state-action cells, each gradient update to the TD target simultaneously shifts the target itself. The agent effectively chases a moving target. Illustrated in Figure 13

To overcome this problem, a frozen copy of the factor matrices is maintained. The TD targets are the computed exclusively from this copy, and is only updated every K steps (e.g. 200). This separates the target from the parameters being optimized, stabilizing training. Introducing a target network improved performance from 0.1 to 1.0 on the empty grid world.

Additionally, it can be argued that this "Deadly Triad" problem also applied to the Phase 2: Deep Q-learning, as the learning curves generally were unstable. This, especially became apparent in the Design 5 and 6 experiments, where the training runs were longer and map sizes were larger - in terms of amount of dimensions in the state space. Originally, this problem was attempted to be solved by changing hyperparameters, such as learning rate, batch size, epsilon decay and replay buffer, which stabilized the learning curves somewhat. Furthermore, during the exploration of good baselines for the second phase on deep q learning, it was observed that the standard DQN implementation would be outperformed by a double DQN implementation. As mentioned in the method: theoretical, the double DQN helps reduce overestimation bias, which is a problem that can lead to unstable learning curves. Here, one could argue, that using the target network approach, would benefit the outcome of the learning curves, as it would further stabilize the learning process. However, due to time constraints (the code having to run over 12 hours each time), this was not implemented in the second phase of the project.

Training time

During training of the models, we noticed that the CP models were taking significantly longer time to train, some cases 10 times longer, compared to tabular q-learning, regardless of the number of ranks used. Some of this was caused by fixable implementation issues. Minor inefficiencies were redundant dense tensor reconstruction for comparing to Q^* , and an unnecessary softmax target computation. This recovered roughly 10-15% of time in total. The dominant cost, however, is structural.

In tabular agent, reading a Q-value is a single array access($O(1)$). In a rank- R CP agent, the same operation requires a dot product across R factor entries ($O(R)$), and computing a gradient of the factor matrices requires leave-one-out products across all N modes, costing $O(NR)$ per update. There is no implementation fix for this, it is the direct tradeoff CP makes in exchange for parameter efficiency. The model stores fewer parameters, but pays a multiplicative cost on every computational cost of the factored representation.

Ethical considerations

The rapid growth of AI model sizes has brought with it a corresponding growth in resource consumption. Memory requirements in particular have scaled so aggressively that some researchers have begun describing the situation as a *memory crisis* [2], where the hardware demands of state-of-the-art models outpace the ability of most institutions to run them. When experimentation requires access to expensive high-memory hardware, the field risks becoming concentrated in the hands of a small number of well-resourced organizations. The work presented in this report is a step in the opposite direction – by representing Q-functions as tensor decompositions, the memory footprint is reduced substantially, in some cases by a factor of 350 or more. This aligns directly with **SDG 12: Responsible Consump-**

tion and Production [5], which calls for more efficient use of resources across all sectors, including computing infrastructure.

However, this memory reduction comes with a downside. The tensor representations require more computation per update, meaning that while tensor models consume less RAM, they can take significantly longer to train. Whether this represents a net environmental benefit depends on the hardware and energy source in use, and parameter count alone is an incomplete representation for environmental impact. This strain is worsened when considering that scaling these methods to real-world applications in robotics, healthcare, or autonomous systems would involve orders of magnitude more computation than the experiments conducted here. Reporting energy consumption alongside performance metrics is increasingly recognised as a responsibility of AI researchers, and is something future work in this area should consider. Furthermore, the training instabilities observed in this project – particularly the collapse of the CP agent back to near-random performance – are manageable in a grid-world setting, but would be unacceptable in systems making consequential decisions in the real world. The ethical obligation to close the gap between research environments and deployment grows as these methods move closer to application.

Conclusion

This project aimed to answer two questions: whether tensor networks can achieve competitive performance with fewer parameters than traditional models, and how different network architectures compare across performance, parameter efficiency, and training stability.

On the first question, our results prove that one can, in some, but not all, circumstances can achieve the same performance with way fewer parameters in grid based RL environments. In phase 1 of this project, it was shown that swapping out a Q-table with CP decomposition base tensors can lead to a parameter reduction at 90% and much faster model train times, whilst keeping similar performance in accuracy. During this experiment it was also discovered that tensor networks perform much better on environments with random agent spawn points as opposed to fixed ones. Phase 2 showed that pure CP (design 1) could outperform a baseline MLP on the Empty grid environment, whilst reducing parameter amount by 350x, however it failed to generalize to more complex environments. Design 2b (tensorized layers) and Design 3 (TT embedding + MLP) both outperformed the baseline on the more complex Distshift environment (78% vs 62%). The answer on the first question therefore depends very much on environment complexity and choice of design.

On the second question, we found that the most important architectural design choice is preserving the original spatial structure of the input. Across all environments, Design 2b consistently outperformed Design 2a across all environments, despite both of them being weight decomposition approaches. The only difference being that Design 2b preserved spatial structure of the input, allowing the decompositions to take full advantage of their multi-dimensional abilities. Design 3 performed well because it also preserved the spatial structure before passing it to the MLP part, outperforming the baseline input flattening MLP.

In addition to answering these two questions, some additional findings are worth noting. After training the CP models with a frozen target network, combating the deadly triad problem, sometimes making models suddenly match the accuracy of tabular Q-learning. Additionally, the CP models parameter efficiency scaled with size, suggesting promising performance gains for grid-based environment larger than 48x48, the biggest configuration in this experiment.

It should be noted that while these experiments showed large performance gains, many of these environments were relatively simple and could potentially be solved by even simpler models than tensor networks. Additionally, it is unclear whether tensor network based models will perform on more complex environments, as something as simple as fixing the spawning point made the models struggle.

Future work could include testing the models on environments with even larger grid sizes as well as more dimensions, since the models scaled well with these changes throughout this project.

References

- [1] Fast simple tensor learning in python.
- [2] Memory crisis hits such extremes that ‘even apple can’t be safe’.
- [3] Minigrid-distshift.
- [4] Minigrid-doorkey.
- [5] The 17 goals.
- [6] Jacob Biamonte. Lectures on quantum tensor networks. *Skolkovo Institute of Science and Technology*, 2020.
- [7] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *Society for Industrial and Applied Mathematics*, 2009.
- [8] Nvidia. What is reinforcement learning?
- [9] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.
- [10] Yannis Panagakis, Jean Kossaifi, Grigorios G. Chrysos, James Oldfield, Mihalios A. Nicolaou, Anima Anandkumar, and Stefanos Zafeiriou. Tensor methods in computer vision and deep learning. *Proceedings of the IEEE*, pages 1–25, 2021.
- [11] Richik Sengupta, Soumik Adhikary, Ivan Oseledets, and Jacob Biamonte. Tensor networks in machine learning. *European Mathematical Society Magazine*, pages 1–8, 2022.
- [12] Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction.

Appendix

GitHub link for code

https://github.com/octyo/tensor_rl

cases/ directory explained

The cases implemented in phase 2 include:

- Case 0/0a: A baseline deep Q-network without any tensor decompositions, serving as a control for comparison. This model consists of a simple feedforward neural network with 1 and 2 hidden layers, respectively. Case 0a is a double DQN version of the baseline, which is a common improvement to the standard DQN algorithm that helps to reduce overestimation bias in action value estimates.
- Case 1: CP factorization of the Q-network. Layers are represented as tensors and decomposed using the CP decomposition method. This approach is expected to reduce the number of parameters in the model, potentially leading to improved efficiency and performance.
- Case 2: For the first subcase A (2a), we implemented a decomposition on the weight matrix pre-initialization, where the weight matrix is decomposed. The second subcase B (2b) is a decomposition on the weight matrix post-initialization, where the weight matrix is decomposed after being initialized as a normal dense layer.
- Case 3: This model tensorizes and decomposes the input to the Q-network, which is an intuitive and easy approach to saving space, while still being able to use the general architecture of the DQN. The input is tensorized based on the dimensions of the game state.
- Case 4: CNN based DQN. Includes *tensorized* CNN which tensorizes the convolutional kernels into the factorized form of the tensor decomposition method. Also includes *backbone* CNN which extracts features with a CNN before moving into fully connected layers, which are then decomposed with the tensor decomposition method.
- Case 5: Replaces the fully connected neural network layers with a tensor layer. Here a tensor layer is defined as a layer that takes a tensor input and produces a tensor output, using tensor operations and decompositions to compute the output.
- Case 6: Replaces the fully connected neural network layers with a tensor network-based matrix product state (MPS) layer. The MPS layer is a specific type of tensor layer that uses the matrix product state decomposition to represent the weights and computations of the layer. This approach is expected to provide a more compact representation of the model, potentially leading to improved efficiency and performance.

After reviewing the code, it was found that, case 1 was not a CP decomposition to tensorize the Q-function, but rather a CP decomposition of the layers in a deep neural network; one has linearity, while the other has non-linearity. Furthermore, case 5 did not correctly spatially tensorize the layers, but rather used the same architecture as case 2a. This means that for cases/, case 1, 2 and 5 are almost entirely similar.

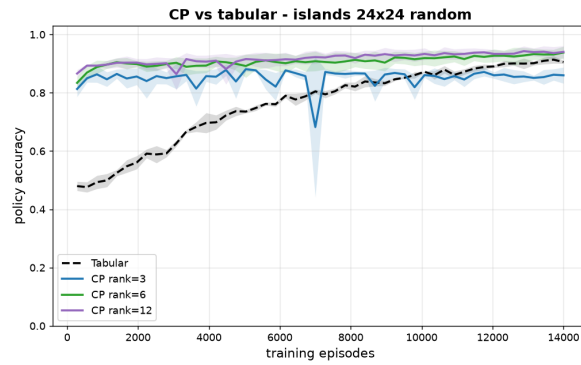


Fig. 12: Caption

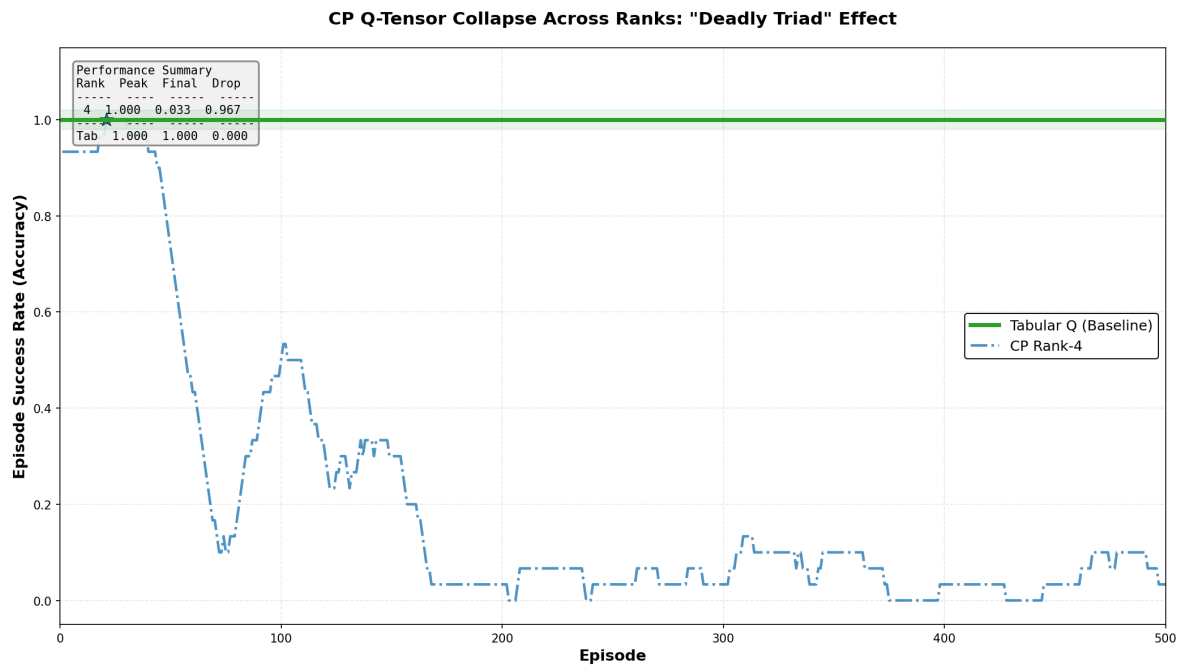


Fig. 13: Visualisation of deadly triad

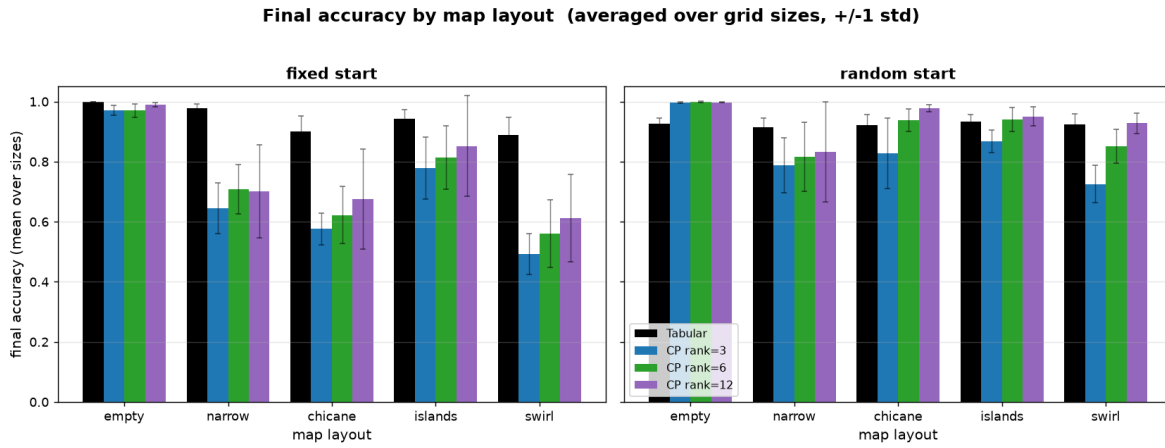


Fig. 14: Final accuracy by different map layouts.

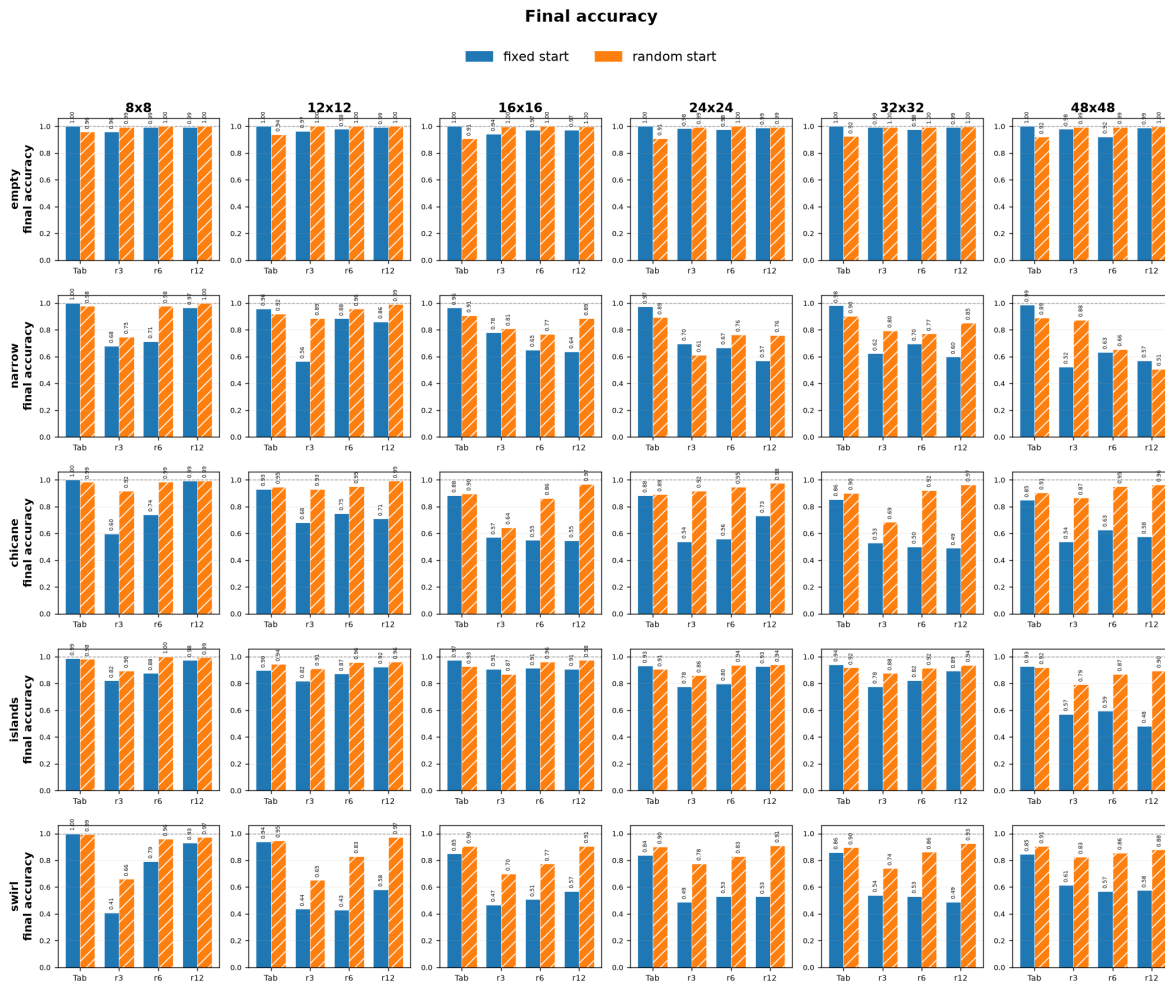


Fig. 15: Final accuracy for phase 1 (tabular vs different rank CP models) across all grid sizes and map layouts.

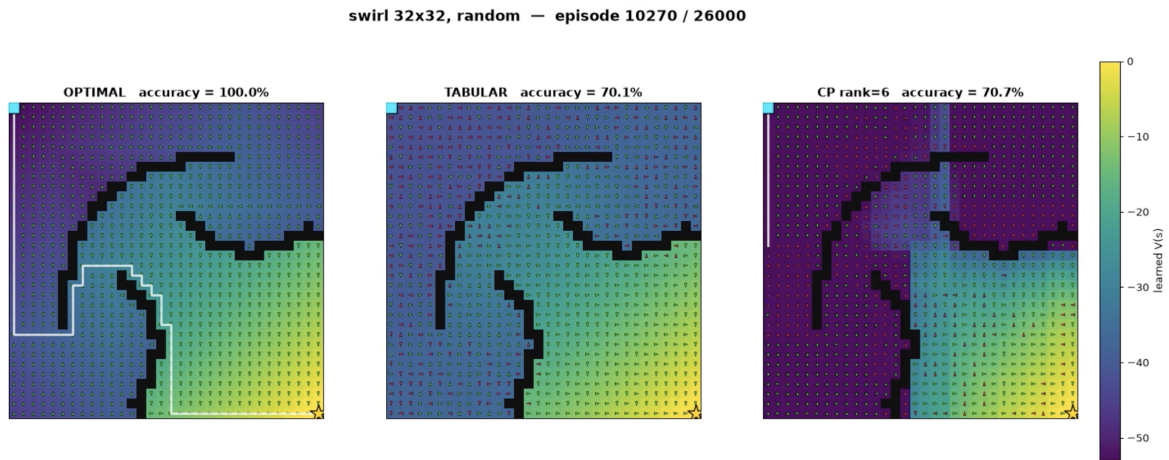


Fig. 16: Caption

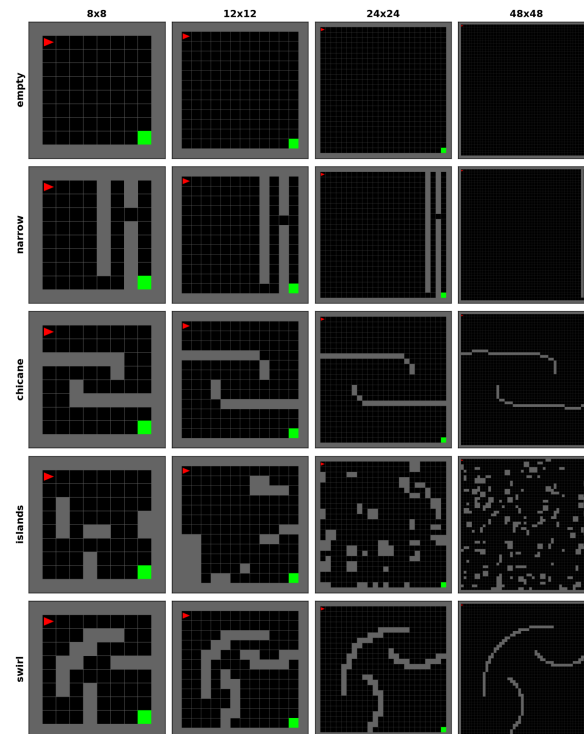
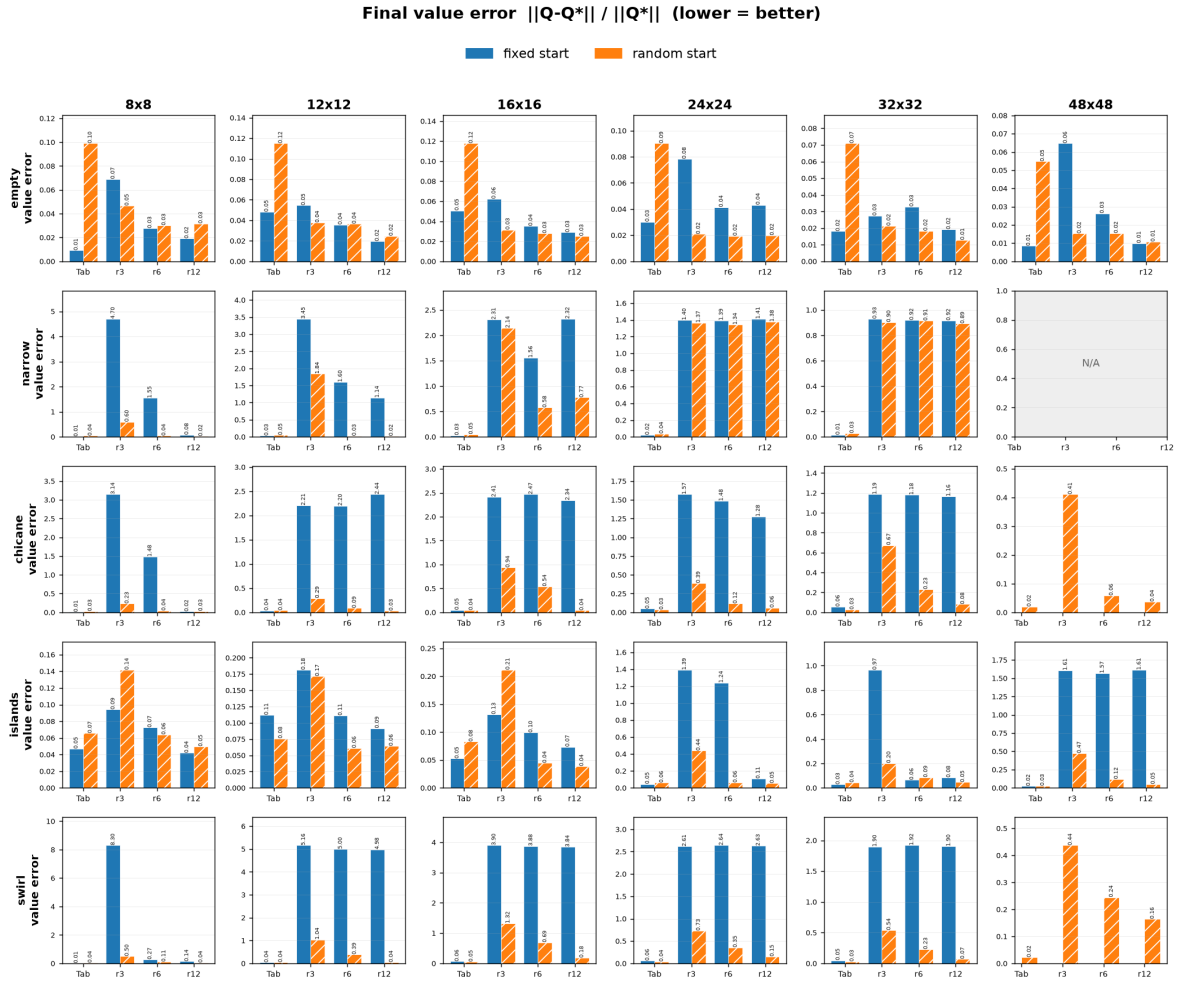


Fig. 17: Caption

Fig. 18: Final accuracy, in value error of Q^*

	Return	Value error	Parameter Ratio
Tabular Baseline	-7.370 ± 0.15	0.073	1.00x
CP Rank 1	-24.890 ± 8.69	128.03	0.14x
CP Rank 2	-7.390 ± 0.14	0.31	0.28x
CP Rank 4	-7.320 ± 0.16	0.13	0.56x
CP Rank 8	-7.430 ± 0.07	0.07	1.12x

More results

Cross-design best solve rates

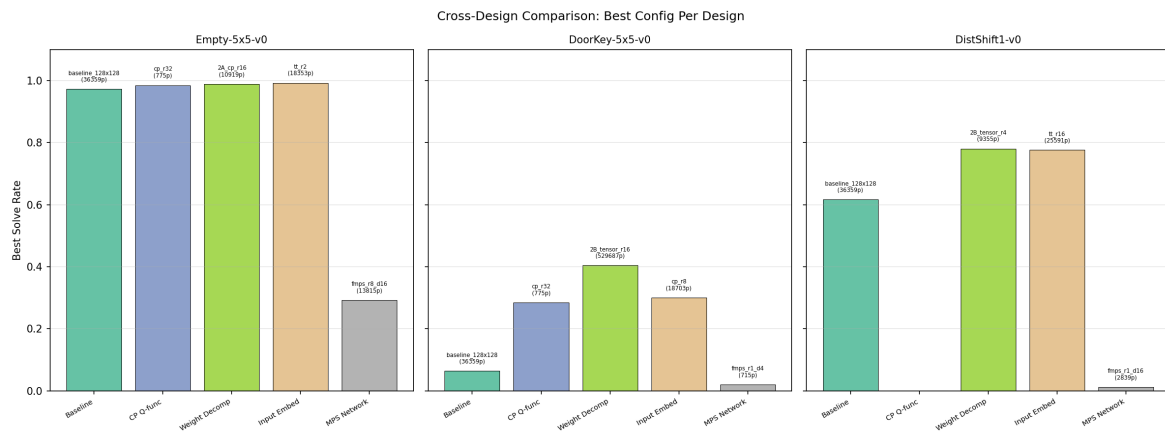


Fig. 19: Best solve rates across all designs and environments.

Baselines across environments

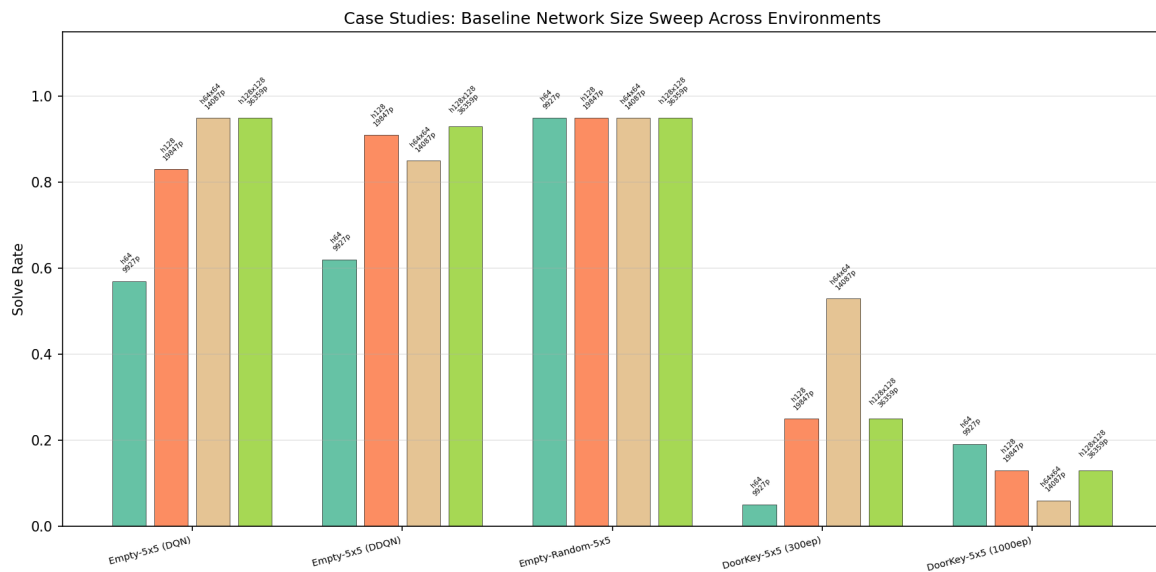


Fig. 20: Baseline performance across environments.

Case 0a: DoorKey-5x5 1000 episodes output

Case 0a rerun (DoorKey-5x5): Baseline Double DQN

env=MiniGrid-DoorKey-5x5-v0, episodes=1000, seeds=[42-48]

Config	Params	Solve%	Reward
h64	9927	19% +/- 29%	0.166 +/- 0.244
h128	19847	13% +/- 21%	0.116 +/- 0.184
h64x64	14087	6% +/- 14%	0.050 +/- 0.122
h128x128	36359	13% +/- 30%	0.106 +/- 0.251

Training convergence (mean reward per 10% bucket):

Config	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
h64	0.120	0.190	0.319	0.307	0.248	0.242	0.263	0.160	0.152	0.153
h128	0.168	0.164	0.243	0.278	0.197	0.204	0.207	0.144	0.125	0.116
h64x64	0.141	0.344	0.335	0.257	0.080	0.027	0.033	0.034	0.041	0.055
h128x128	0.177	0.347	0.309	0.203	0.121	0.062	0.091	0.100	0.111	0.104

Max solve rate observed during training: 34% (h128x128 at 20% bucket)

Total runtime: 215.8 min

CP Q-function: learning curves (Empty-5x5)

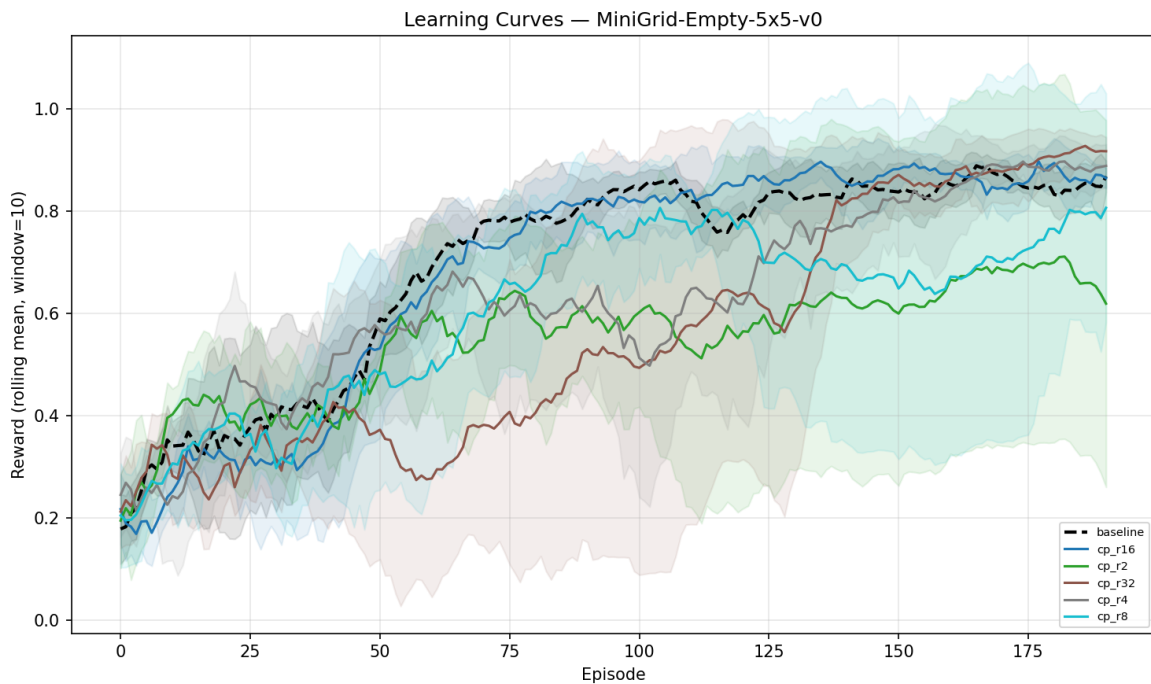


Fig. 21: Learning curves for Design 1 (CP) on Empty-5x5.

CP Q-function: parameter efficiency (Empty-5x5)

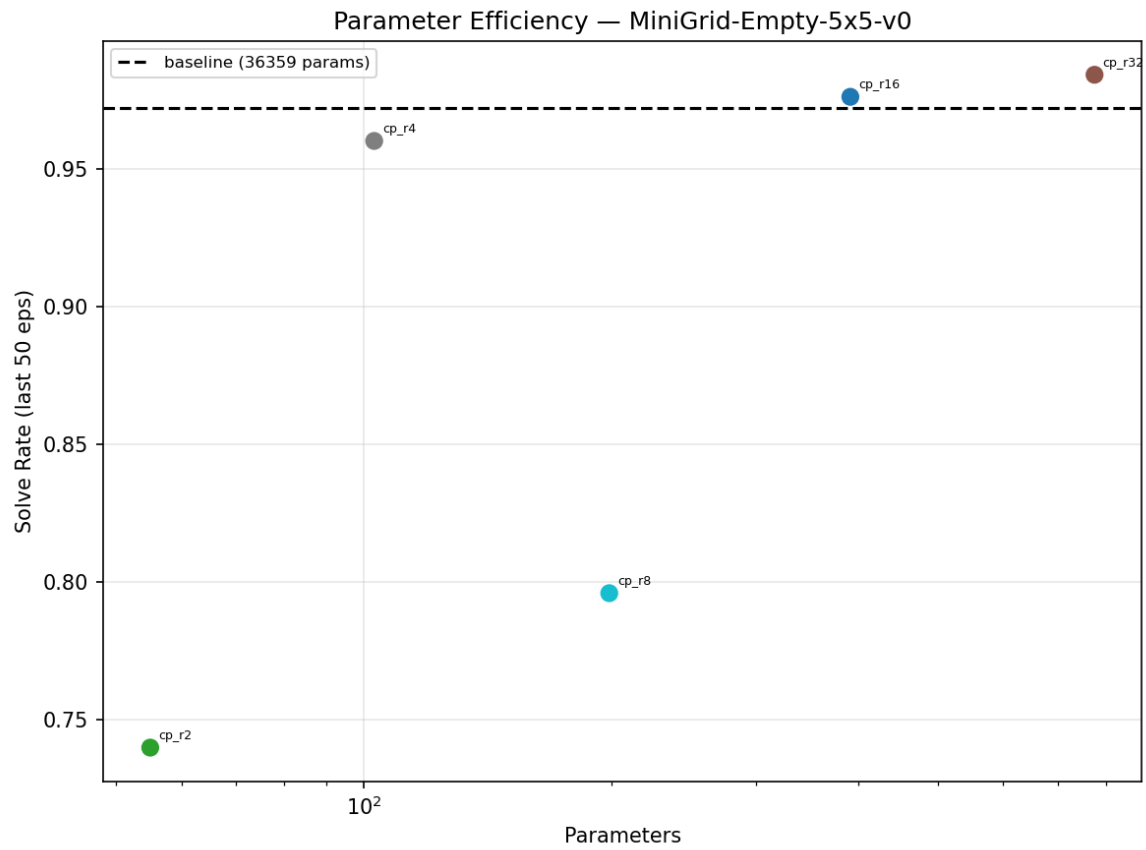


Fig. 22: Parameter efficiency for Design 1 (CP) on Empty-5x5.

CP Q-function: learning curves (DoorKey and DistShift)

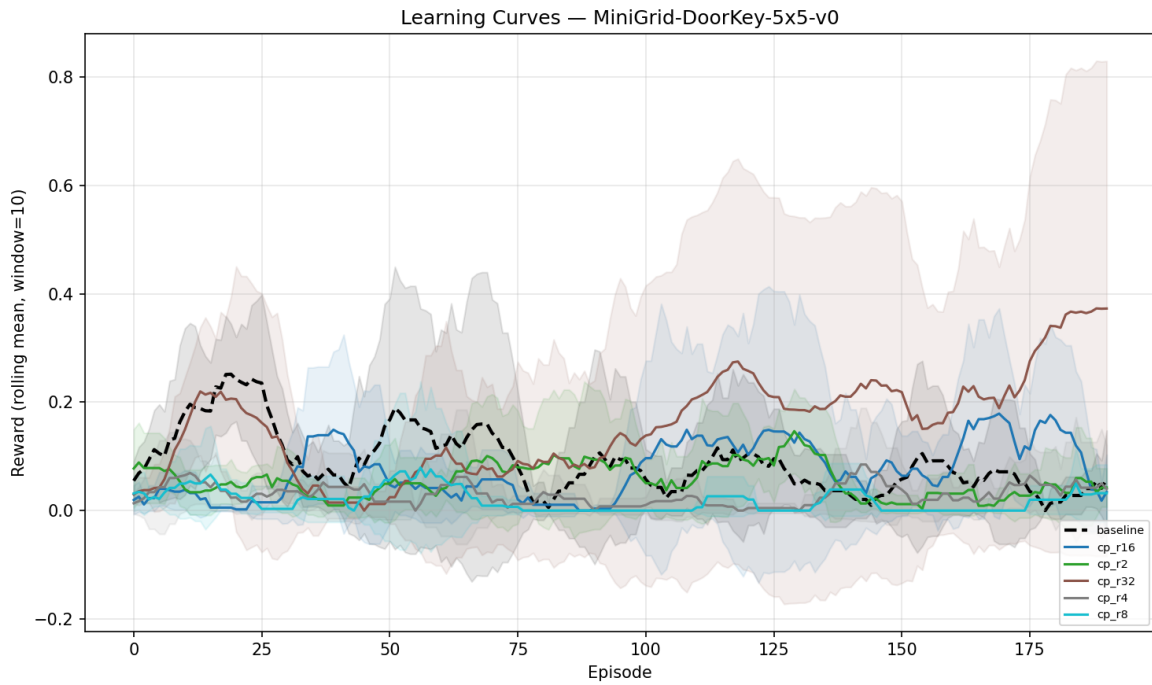


Fig. 23: Learning curves for Design 1 (CP) on DoorKey-5x5.

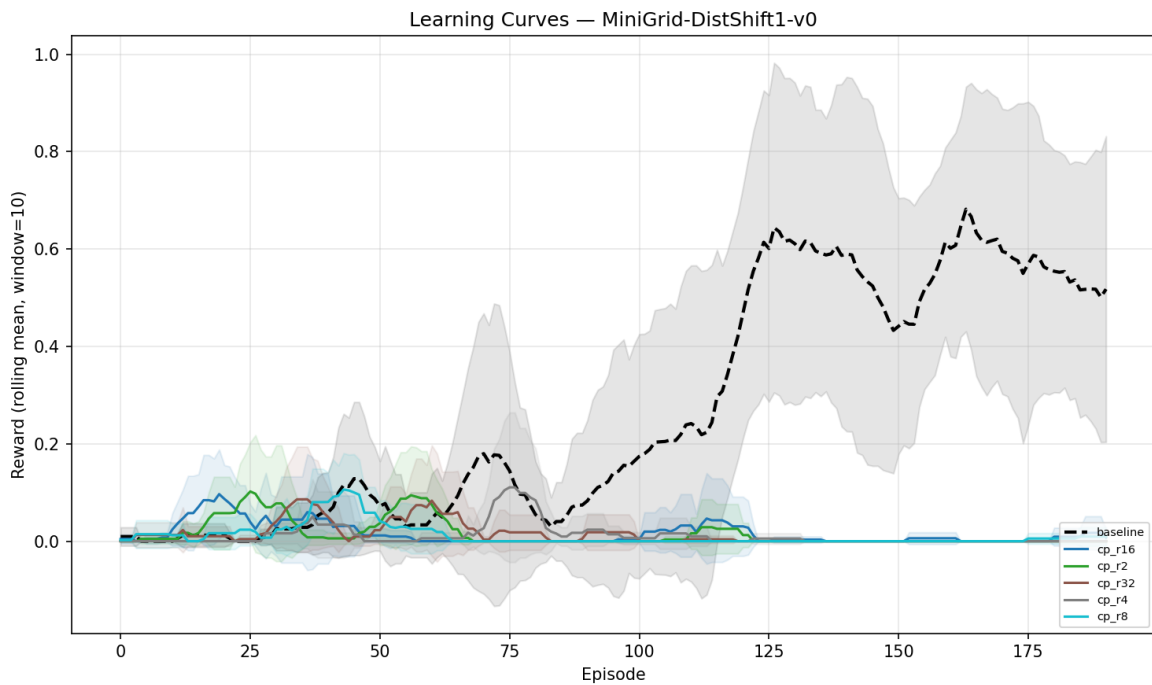


Fig. 24: Learning curves for Design 1 (CP) on DistShift1.

Weight decomposition: parameter efficiency (DistShift1)

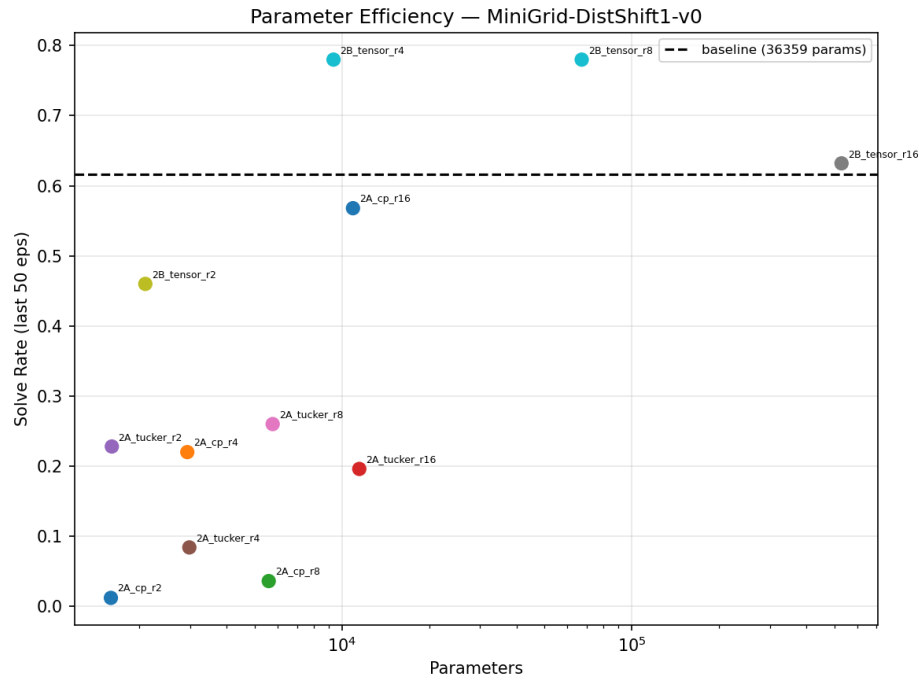


Fig. 25: Parameter efficiency for Design 2 on DistShift1.

Weight decomposition: convergence heatmap (DistShift1)

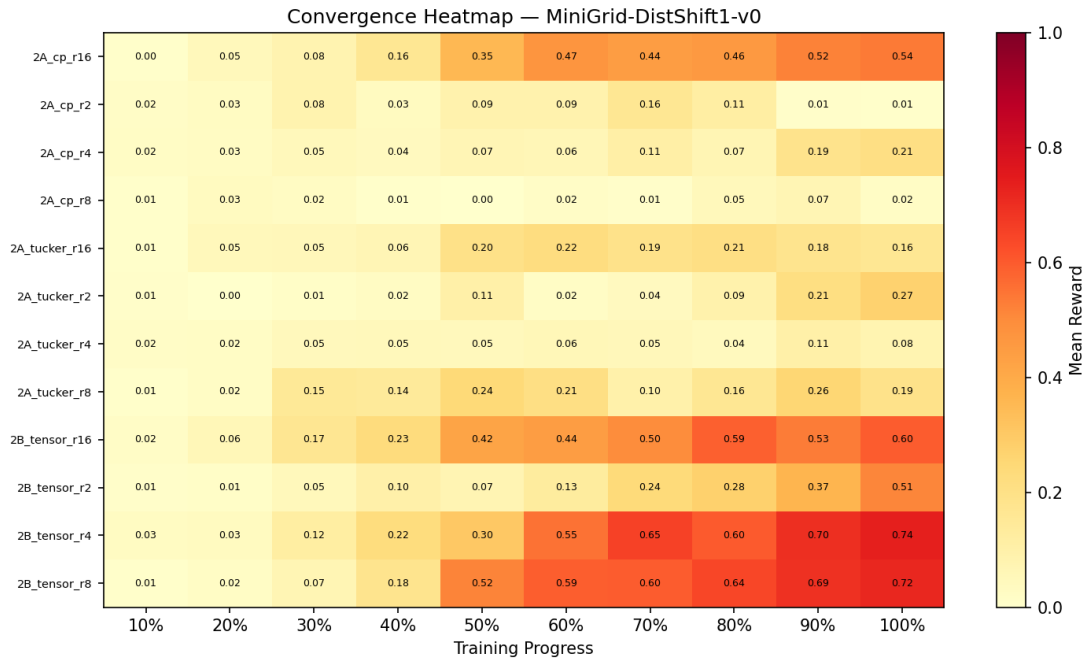


Fig. 26: Convergence heatmap for Design 2 on DistShift1.

Weight decomposition: parameter efficiency (DoorKey-5x5)

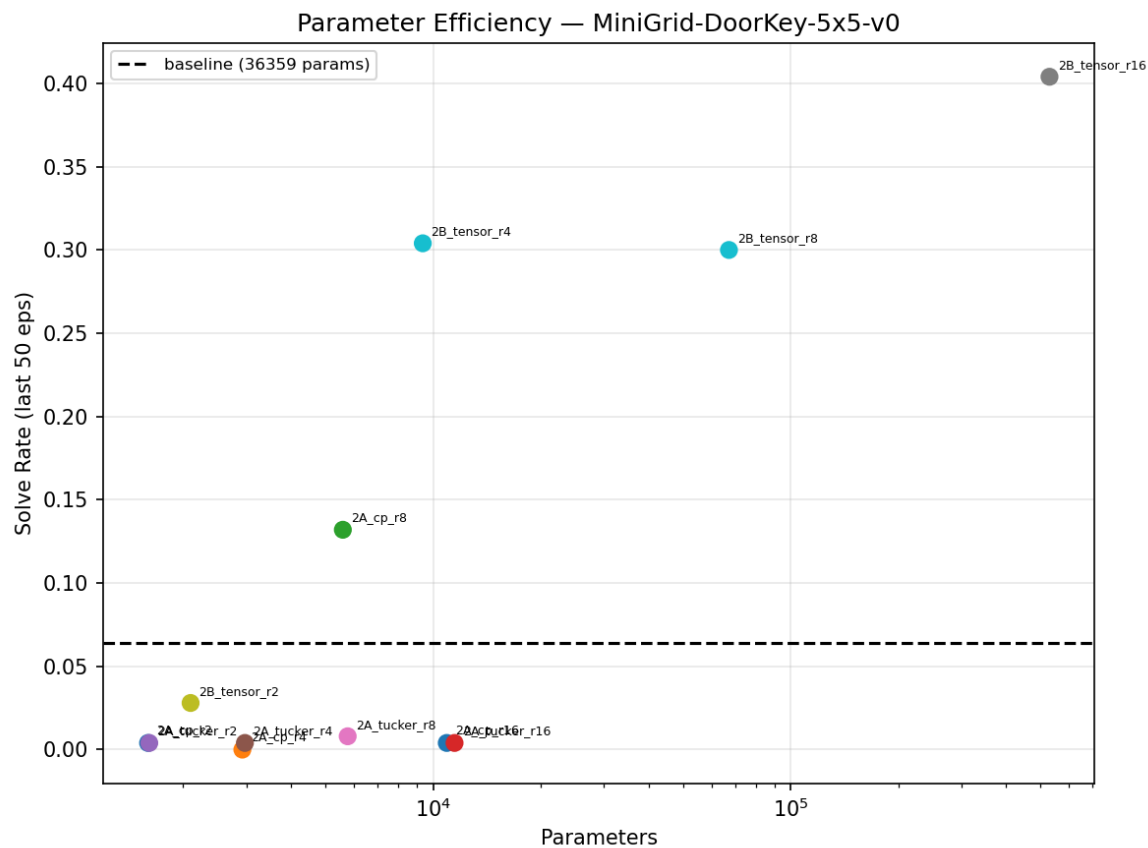


Fig. 27: Parameter efficiency for Design 2 on DoorKey-5x5.

Weight decomposition: parameter efficiency (Empty-5x5)

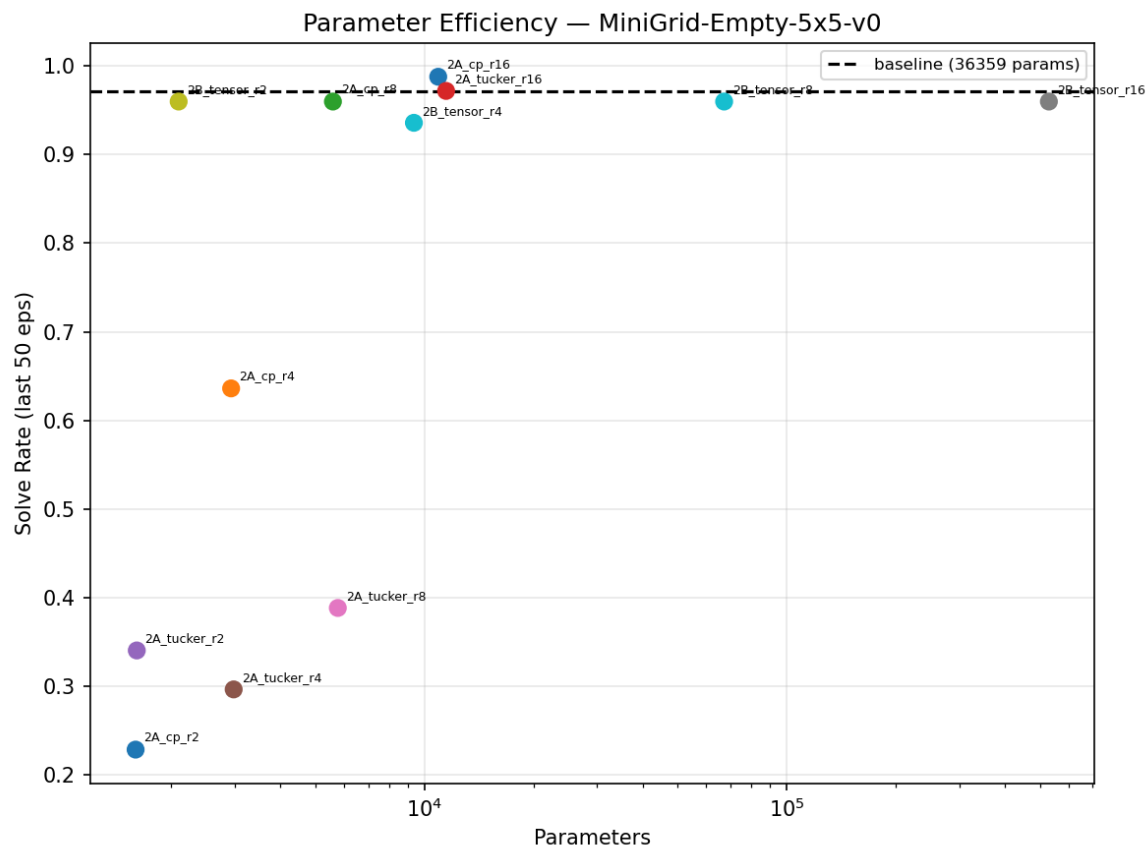


Fig. 28: Parameter efficiency for Design 2 on Empty-5x5.

Input reduction: convergence heatmap (DistShift1)

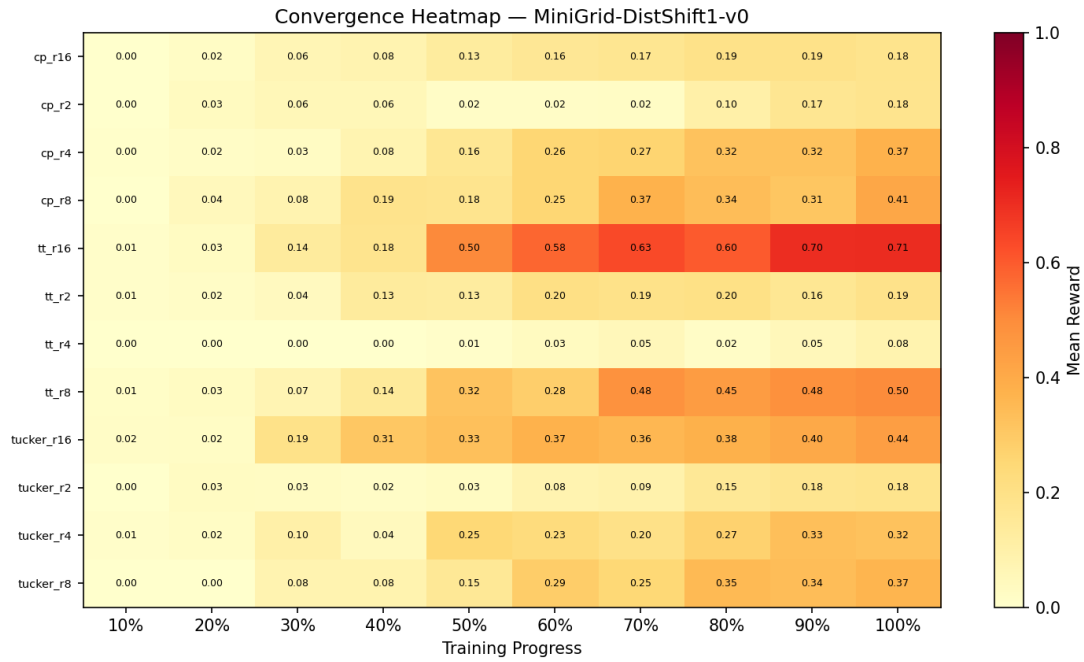


Fig. 29: Convergence heatmap for Design 3 on DistShift1.

MPS: convergence heatmap (Empty-5x5)

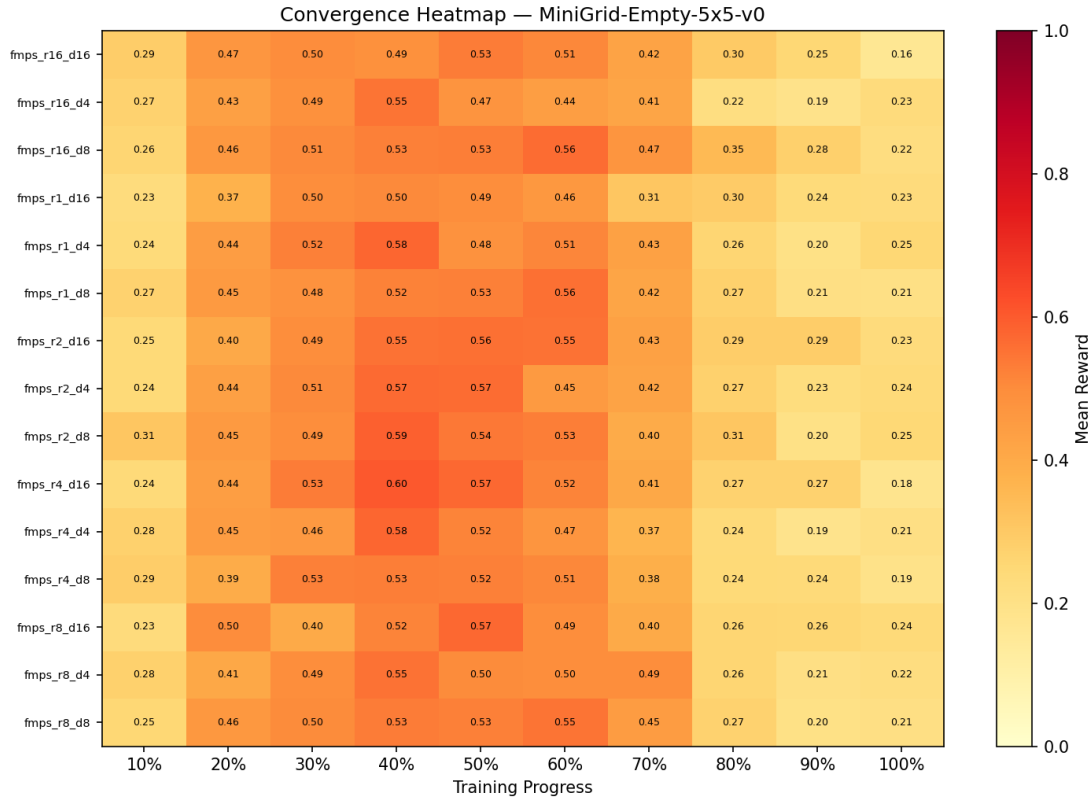


Fig. 30: Convergence heatmap for Design 4 (MPS) on Empty-5x5.

Parameter counts for all designs

All parameter counts are computed for the MiniGrid environment with observation shape $(7, 7, 3)$ giving $I = 147$ flat dimensions, $N_a = 7$ actions, and mode dimensions $d_1 = 7, d_2 = 7, d_3 = 3$.

Design	Config	Params	Formula
<i>Design 0: Baseline MLP</i>			
	$P = (I \cdot h_1 + h_1) + (h_1 \cdot h_2 + h_2) + (h_2 \cdot N_a + N_a)$		
	128 × 128	36,359	
<i>Design 1: Pure CP Q-function</i>			
	$P = R \cdot (d_1 + d_2 + d_3 + N_a) + N_a$		
	Rank 1	31	
	Rank 2	55	
	Rank 4	103	
	Rank 8	199	
	Rank 16	391	
	Rank 32	775	
<i>Design 2A: CP weight decomp (flat)</i>			
	Per layer: $P_l = R \cdot (I_{in} + I_{out}) + I_{out}$. Total: $\sum P_l + (h_2 \cdot N_a + N_a)$		
	cp_r2	2,221	
	cp_r4	3,283	
	cp_r8	5,407	
	cp_r16	9,655	
<i>Design 2A: Tucker weight decomp (flat)</i>			
	Per layer: $P_l = R^2 + R \cdot (I_{in} + I_{out}) + I_{out}$. Total: $\sum P_l + (h_2 \cdot N_a + N_a)$		
	tucker_r2	2,229	
	tucker_r4	3,315	
	tucker_r8	5,535	
	tucker_r16	10,167	
<i>Design 2A: TT weight decomp (flat)</i>			
	Per layer: $P_l = I_{out} \cdot R + R \cdot I_{in} + I_{out}$. Total: $\sum P_l + (h_2 \cdot N_a + N_a)$		
	tt_r2	2,221	
	tt_r4	3,283	
	tt_r8	5,407	
	tt_r16	9,655	
<i>Design 2B: Tensorized layer (structured input)</i>			
	$P = \sum_n d_n \cdot R + R^N + (R^N \cdot h + h) + (h \cdot N_a + N_a)$		
	r2	2,097	
	r4	9,355	
	r8	67,215	
	r16	529,687	
<i>Design 3: CP embedding + MLP</i>			
	Emb: $R \cdot h + h + R \cdot \sum_n d_n$. MLP: $(h \cdot h + h) + (h \cdot N_a + N_a)$		
	cp_r2	17,833	
	cp_r4	18,123	
	cp_r8	18,703	
	cp_r16	19,863	
<i>Design 3: TT embedding + MLP</i>			
	Emb: $1 \cdot d_1 \cdot R + R \cdot d_2 \cdot R + R \cdot d_3 \cdot h + h$. MLP: $(h \cdot h + h) + (h \cdot N_a + N_a)$		
	tt_r2	18,353	
	tt_r4	19,219	
	tt_r8	21,119	
	tt_r16	25,591	
<i>Design 4: FeatureMap MPS</i>			
	Cores: $1 \cdot d_p \cdot r + (n-2) \cdot r \cdot d_p \cdot r + r \cdot d_p \cdot N_a + N_a$. Emb: $\sum_k g_k \cdot d_p + n \cdot d_p$		
	r1_d4	715	
	r1_d8	1,423	
	r1_d16	2,839	
	r2_d4	867	
	r2_d8	1,727	
	r2_d16	3,447	
	r4_d4	1,411	
	r4_d8	2,815	
	r4_d16	5,623	
	r8_d4	3,459	
	r8_d8	6,911	
	r8_d16	13,815	
	r16_d4	11,395	
	r16_d8	22,783	